

Mismatch-Free One-Step at Scale: Linear-Time Ensemble Clustering with Soft Balancing

Haoye Qiu
Southeast University
Nanjing, China
qhy@seu.edu.cn

Xu Zhang
Southeast University
Nanjing, China
xuz@seu.edu.cn

Ding Ding
Southeast University
Nanjing, China
dingding-1@seu.edu.cn

Yuheng Jia*
Southeast University
Nanjing, China
yhjia@seu.edu.cn

Abstract—Ensemble clustering integrates multiple base clusterings into a more reliable consensus result; however, co-association matrix-based methods have high overhead and limited scalability on large datasets. Although scalable ensemble clustering has been developed, existing methods still face two challenges: (i) multi-step pipelines may incur information loss and suffer from both cluster-number and cluster-correspondence mismatches; and (ii) discrete label learning may induce cluster-size bias at the representation and optimization levels, yielding highly imbalanced degenerate partitions. To this end, we fill this gap by proposing One-Step Linear-Time Ensemble Clustering with Soft Balancing (OLEC-SB), a unified framework that integrates discrete label learning and theoretically-grounded soft balancing in linear time to address the above two challenges. For Challenge 1, OLEC-SB updates the discrete indicator matrix on a sample-cluster bipartite representation. Thus, labels are obtained without connected-component detection, and explicit cluster-index alignment is avoided by measuring super-node similarity from pairwise co-occurrence relations. For Challenge 2, OLEC-SB incorporates cluster-size information into the normalized indicator matrix and introduces balanced regularization to alleviate the “rich-get-richer” effect (*i.e.*, larger clusters become increasingly dominant). Built upon super-node similarity minimization with adaptive weighting, OLEC-SB captures the importance of different base clusterings and learns discrete labels in a one-step manner. To solve the discrete problem efficiently, we further develop a fast coordinate descent algorithm with three acceleration strategies, achieving linear complexity in the number of samples. Extensive experiments on various datasets demonstrate that OLEC-SB outperforms state-of-the-art methods in terms of both effectiveness and efficiency. The code and supplementary material are available at <https://github.com/haoyeqiu/OLEC-SB>.

Index Terms—Ensemble clustering, large-scale data, discrete label learning, soft balancing

I. INTRODUCTION

Ensemble clustering integrates multiple base clusterings into a unified partition, and produces a more accurate and robust consensus result than any single clustering [1]–[3]. Since it does not require access to original data features, it is suitable for privacy-preserving analysis and secure data utilization [4]. Most existing methods construct a co-association (CA) matrix [5] from all base clusterings, where each entry records how frequently two samples are assigned to the same cluster. To improve the CA matrix, various methods [6]–[9] refine pairwise co-occurrence relations, and obtain the consensus partition by hierarchical or spectral clustering. However, the

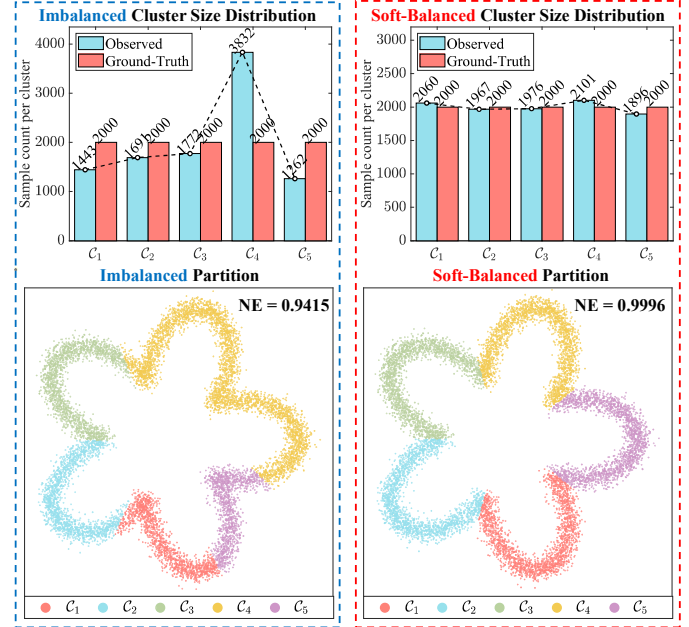


Fig. 1. Illustration of cluster-size bias and soft balancing. Without soft balancing, discrete label learning may suffer from a “rich-get-richer” effect and produce highly imbalanced partitions (left). With soft balancing, OLEC-SB encourages a more even cluster-size distribution and a more reasonable partition (right). A larger NE [10] indicates better cluster balance.

$n \times n$ -sized CA matrix needs $\mathcal{O}(n^2)$ memory and often $\mathcal{O}(n^2)$ or even $\mathcal{O}(n^3)$ time, which limits scalability on large-scale datasets. To solve the problem, much effort has been dedicated to improving the scalability of ensemble clustering, including random sample partition (RSP) [11], sparse representation [12] and anchor strategy [13], [14]. In particular, RSP-based methods [15], [16] perform clustering on a subset of randomly sampled blocks and then merge the resulting centroids to form the consensus partition. Sparse-representation methods [17]–[19] compress sample-level co-occurrence into representative micro-clusters to avoid the quadratic cost of sample-level CA matrices. Anchor-based methods [20]–[22] select representative samples (anchors) to construct a sample–anchor similarity matrix, from which a spectral embedding is obtained and then discretized by k -means. Although these methods are efficient, the multi-step pipeline may incur information loss [23].

*Yuheng Jia is the corresponding author.

Recently, several one-step scalable methods [24]–[26] have been proposed. These methods impose a Laplacian rank constraint on a bipartite graph and obtain discrete labels from connected components via Tarjan’s algorithm [27]. However, such bipartite formulations may suffer from two mismatch issues: (i) the detected connected components may not match the underlying cluster structure, leading to cluster-number mismatch; and (ii) cluster indices are not naturally aligned across different base clusterings, leading to cluster-correspondence mismatch. Moreover, discrete label learning may induce two-level cluster-size bias: (i) CA matrices tend to bias ambiguous samples toward larger clusters; and (ii) row-wise updates of the discrete indicator matrix may be biased toward larger candidate clusters when their average similarities are close. **These limitations motivate a unified one-step ensemble framework for discrete label learning in linear time, avoiding mismatch issues and correcting cluster-size bias.** Specifically, we pose two challenges:

Challenge 1: Mismatch-Free One-Step Ensemble at Scale.

Existing one-step scalable methods suffer from cluster-number and cluster-correspondence mismatches. Some studies [10], [28] introduce learnable permutation or orthogonal rotation matrices for cluster-index alignment, but such explicit alignment becomes difficult when base clusterings have different cluster numbers. A simple alternative is to concatenate base indicator matrices, but it assumes all base clusterings contribute equally. *The challenge is to output discrete labels with linear complexity in a one-step manner while avoiding these mismatch issues.*

Challenge 2: Cluster-Size Bias in Discrete Label Learning.

In Figure 1, ambiguous samples in the overlap between $C3$ and $C4$ are assigned to the larger $C4$ due to the representation-level bias of CA matrices. This bias is further amplified at the optimization level, where row-wise updates make $C4$ attract more samples than the smaller $C5$ and become increasingly dominant, yielding a “rich-get-richer” effect and a highly imbalanced partition. *The challenge is to mitigate such bias via soft balancing while preserving accuracy and efficiency.*

To address these challenges, we fill the gap and propose a unified framework named **One-Step Linear-Time Ensemble Clustering with Soft Balancing (OLEC-SB)**. The key idea is to formulate a soft-balanced consensus objective on a sample-cluster bipartite representation and optimize the discrete indicator matrix within a mismatch-free one-step ensemble framework. In this formulation, discrete labels are obtained without connected-component detection, and super-node similarity between each cluster and its complement is measured from pairwise co-occurrence relations to avoid explicit cluster-index alignment, addressing the mismatch issues in Challenge 1. Specifically, we first construct a normalized indicator matrix with cluster-size information to mitigate representation-level bias in CA matrices. Based on this representation, we introduce adaptive weighting to capture the importance of different base clusterings and optimize the discrete indicator matrix by minimizing super-node similarity in a one-step manner. To address optimization-induced bias, we further introduce

balanced regularization to alleviate the “rich-get-richer” effect and prevent degenerate partitions. In this way, OLEC-SB mitigates the two-level cluster-size bias in Challenge 2. Finally, we develop a fast coordinate descent (CD) algorithm with three acceleration strategies, achieving an overall complexity of $\mathcal{O}(nm + t(nKmt_Y + nm))$, where n , K , m , t , and t_Y denote the number of samples, clusters, base clusterings, outer iterations, and inner CD iterations, respectively. Since $K, m, t, t_Y \ll n$, the complexity is linear in n . Extensive experiments on various datasets show that OLEC-SB outperforms state-of-the-art (SOTA) methods in both effectiveness and efficiency. Our main contributions are as follows:

- **Effectiveness:** OLEC-SB unifies discrete label learning and soft balancing within a mismatch-free one-step ensemble framework. **To the best of our knowledge, this is the first work on scalable ensemble clustering to address both mismatch issues and two-level cluster-size bias.** Extensive experiments demonstrate its superior clustering performance over SOTA methods.
- **Efficiency:** Instead of constructing an $n \times n$ -sized CA matrix, OLEC-SB operates on a normalized indicator matrix with cluster-size information. Based on this representation, we design a fast CD optimization algorithm with three acceleration strategies, achieving an overall complexity of $\mathcal{O}(nm + t(nKmt_Y + nm))$, which is linear in n when $K, m, t, t_Y \ll n$. Thus, OLEC-SB is scalable for large-scale ensemble clustering.
- **Theoretical Analysis:** We prove the doubly stochastic property of the normalized CA matrix, the monotonic convergence of OLEC-SB, and its averaged normalized-cut interpretation, and further show how balanced regularization corrects optimization-induced cluster-size bias in row-wise updates.

II. RELATED WORK

In this section, we first introduce ensemble clustering and the co-association (CA) matrix, and then review scalable ensemble clustering and balanced clustering. The main notations used in this paper are listed in Table I.

A. Ensemble Clustering and CA Matrix

Let $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$ be a dataset of n samples and $\Pi = \{\pi_1, \dots, \pi_m\}$ a set of m base clusterings. For the k -th base clustering π_k with l_k clusters, let $\mathbf{A}_k \in \{0, 1\}^{n \times l_k}$ denote its indicator matrix, where $(\mathbf{A}_k)_{ir} = 1$ if $\mathbf{x}_i$ belongs to its r -th cluster and 0 otherwise. The co-association (CA) matrix is $\mathbf{S} = \frac{1}{m} \sum_{k=1}^m \mathbf{A}_k \mathbf{A}_k^\top$, where S_{ij} is the co-occurrence frequency of $\mathbf{x}_i$ and $\mathbf{x}_j$ across base clusterings. A consensus partition is obtained by hierarchical or spectral clustering on $\mathbf{S}$ [29], [30].

B. Scalable Ensemble Clustering

Scalable ensemble clustering methods [31] can be grouped into three categories: random sample partition (RSP), sparse representation and anchor strategy. RSP-based methods [11], [15], [16] split data into random blocks, perform clustering on a subset of blocks, and merge the resulting centroids to obtain

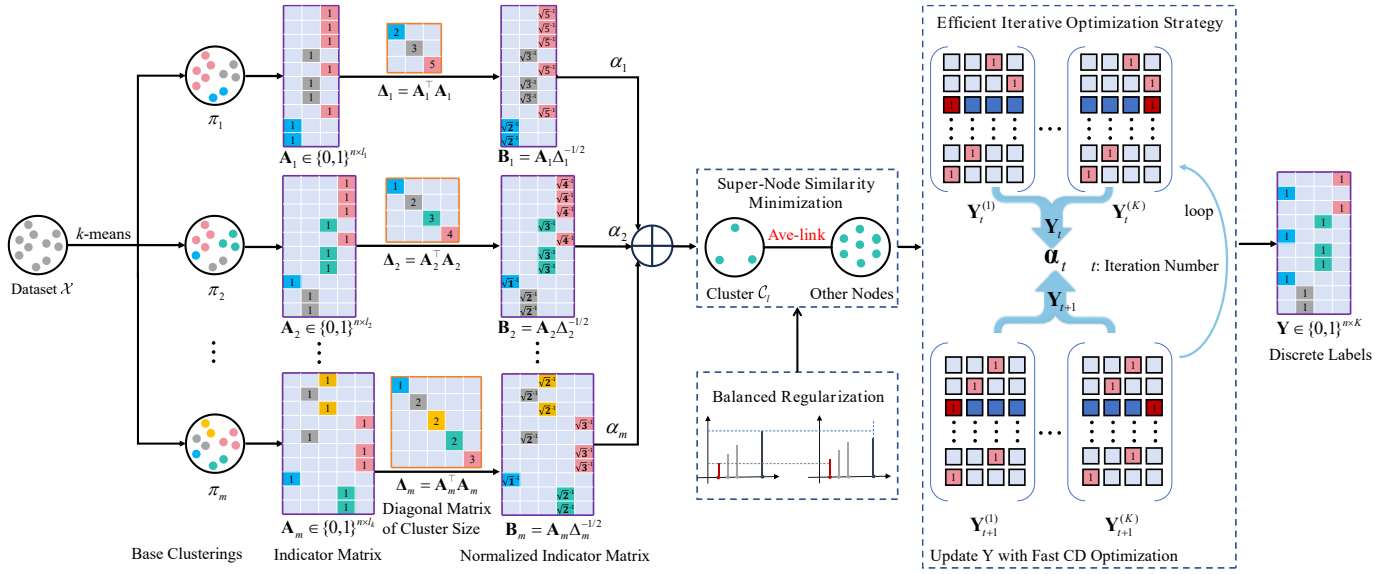


Fig. 2. Overall framework of OLEC-SB. We first generate m base clusterings $\{\pi_k\}_{k=1}^m$ by running k -means multiple times. For each base clustering π_k , we construct an indicator matrix $\mathbf{A}_k \in \{0, 1\}^{n \times l_k}$, where n and l_k denote the numbers of samples and clusters in π_k , respectively. Then, cluster-size information is incorporated by $\Delta_k = \mathbf{A}_k^\top \mathbf{A}_k$, yielding the normalized representation $\mathbf{B}_k = \mathbf{A}_k \Delta_k^{-1/2}$. Based on the weighted bipartite representation, OLEC-SB minimizes super-node similarity with balanced regularization and obtains discrete labels $\mathbf{Y} \in \{0, 1\}^{n \times K}$ via fast coordinate descent (CD) optimization, where K is the number of clusters in the consensus partition.

TABLE I
MAIN NOTATIONS.

Notation	Description
n, d, m, K, l_k	Numbers of samples, features, base clusterings, consensus clusters, and k -th base clusters.
$\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$	Input dataset consisting of n samples.
$\Pi = \{\pi_1, \dots, \pi_m\}, \pi_k$	Set of m base clusterings and its k -th member.
$\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}, \mathcal{C}_l$	Consensus partition and its l -th cluster.
$\mathbf{A}_k \in \{0, 1\}^{n \times l_k}, (\mathbf{a}_k)_i$	Indicator matrix of π_k and its i -th row.
$\Delta_k = \mathbf{A}_k^\top \mathbf{A}_k$	Diagonal matrix of cluster sizes in π_k .
$\mathbf{B}_k = \mathbf{A}_k \Delta_k^{-1/2}, (\mathbf{b}_k)_i$	Normalized indicator matrix of π_k and i -th row.
$\mathbf{W}_k = \mathbf{B}_k \mathbf{B}_k^\top$	Normalized CA matrix of π_k .
$\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_K], \mathbf{y}_l$	Clustering indicator matrix and its l -th column.
$\alpha = [\alpha_1, \dots, \alpha_m]^\top, \alpha_k$	Base clustering weight vector and weight of π_k .
$\mathbf{Y}^{(s)}, \mathbf{Y}^{(0)}, p, q, s$	Candidate and auxiliary matrices, with current, updated, and candidate cluster indices.
$\delta_i, \varphi(s)$	One-hot vector with the i -th entry equal to 1, and objective difference of assigning $\mathbf{x}_i$ to $\mathcal{C}_s$.

the consensus partition. However, sampled blocks may be less representative and miss global structure, yielding biased or unstable results. Sparse-representation methods [12], [17]–[19] compress sample-level co-occurrence into representative micro-clusters, which may overlook local co-occurrence information and propagate compression errors. Anchor-based methods [13], [14], [20]–[22] select representative samples (anchors) to approximate the data structure. They learn a sample–anchor similarity matrix, obtain a spectral embedding, and apply k -means to produce cluster labels. However, this multi-step pipeline may incur information loss and suffer from the initialization sensitivity of k -means. To address these issues, recent studies [24]–[26], [28] impose a Laplacian

rank constraint on a bipartite graph to obtain exactly K connected components, with labels obtained by connected-component detection (e.g., Tarjan’s algorithm [27]). **However, they still involve spectral embedding and the number of connected components does not always match the true cluster number.** Recently, Bian et al. [32] proposed OS-FECM, which learns an embedding with the true number of clusters. Although it avoids cluster-number mismatch, OS-FECM is designed for fuzzy ensemble clustering and still needs extra discretization to obtain discrete labels. **Moreover, existing scalable methods often overlook cluster-correspondence mismatch and cluster-size bias in discrete label learning.**

C. Balanced Clustering

Balanced clustering aims to produce roughly equal cluster sizes and is generally divided into hard-balanced and soft-balanced methods. Hard-balanced methods [33], [34] impose explicit size constraints on each cluster, while soft-balanced methods add balanced regularization to the objective [35], [36], offering more flexibility when strict balance is unnecessary. In ensemble clustering, however, soft balancing remains underexplored, and only a few works [10], [37] explicitly incorporate balanced regularization into the consensus objective. Specifically, [10] jointly optimized the indicator matrix and base-clustering weights with balanced regularization, but relied on $n \times n$ CA matrices, requiring $\mathcal{O}(n^2 K)$ to update the indicator matrix and $\mathcal{O}((n^2 K + m^3)t)$ overall complexity. [37] introduced equal-capacity regularization, but its solver still requires $\mathcal{O}(n^2 K^2)$ complexity. **These limitations motivate scalable ensemble clustering that achieves balanced partitioning without quadratic memory or time overhead.**

III. THE PROPOSED METHOD

We propose **One-Step Linear-Time Ensemble Clustering** with **Soft Balancing** (OLEC-SB) to address both mismatch issues and two-level cluster-size bias in discrete label learning. Fig. 2 illustrates the overall framework, and the main notations of OLEC-SB are summarized in the supplementary material.

A. A Unified Mismatch-Free One-Step Ensemble Framework

Given the dataset $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$ and base clusterings $\Pi = \{\pi_1, \dots, \pi_m\}$, suppose $\mathcal{X}$ is partitioned into K disjoint clusters $\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, satisfying: (i) $\bigcup_{l=1}^K \mathcal{C}_l = \mathcal{X}$, (ii) $\mathcal{C}_l \neq \emptyset$ for $l = 1, \dots, K$, and (iii) $\mathcal{C}_l \cap \mathcal{C}_h = \emptyset$ for $l \neq h$. For the k -th base clustering π_k , we construct a weighted undirected graph $\mathcal{G}_k = \{\mathcal{X}, \mathbf{W}_k\}$, where $\mathbf{W}_k \in \mathbb{R}^{n \times n}$ is the pairwise similarity matrix and $w_{ij}^k \geq 0$ measures the similarity between $\mathbf{x}_i$ and $\mathbf{x}_j$. We evaluate a partition on $\mathcal{G}_k$ by measuring the similarity between each cluster $\mathcal{C}_l$ and its complement $\bar{\mathcal{C}}_l = \mathcal{X} \setminus \mathcal{C}_l$. To obtain well-separated clusters, we minimize the similarity between $\mathcal{C}_l$ and $\bar{\mathcal{C}}_l$. Inspired by hierarchical clustering [38], we regard them as two super nodes and define their similarity by summing pairwise similarities across the two sets:

$$\text{sim}_k(\mathcal{C}_l, \bar{\mathcal{C}}_l) = \sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) = \sum_{i \in \mathcal{C}_l} \sum_{j \notin \mathcal{C}_l} w_{ij}^k. \quad (1)$$

However, directly minimizing Eq. (1) may produce excessively large or small clusters. To alleviate this issue, we introduce a balancing term $f(|\mathcal{C}_l|)$ as a cluster-size constraint to discourage such extreme sizes. Considering all clusters, the objective for the k -th base clustering is formulated as

$$\begin{aligned} \min_{\mathcal{C}} \quad & \sum_{l=1}^K \sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) + \lambda \sum_{l=1}^K f(|\mathcal{C}_l|) \\ \text{s.t.} \quad & \bigcup_{l=1}^K \mathcal{C}_l = \mathcal{X}, \mathcal{C}_l \neq \emptyset, \mathcal{C}_l \cap \mathcal{C}_h = \emptyset, \end{aligned} \quad (2)$$

where $\lambda > 0$ controls the trade-off between the balancing term and the inter-cluster similarity term: a larger λ penalizes highly imbalanced cluster sizes more aggressively, while a smaller λ places more emphasis on minimizing the inter-cluster similarity term.

To reflect the importance of each base clustering, we introduce a nonnegative weight α_k for the k -th base clustering. Consequently, problem (2) can be reformulated as

$$\begin{aligned} \min_{\mathcal{C}, \alpha} \quad & \sum_{k=1}^m \alpha_k^\gamma \sum_{l=1}^K \sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) + \lambda \sum_{l=1}^K f(|\mathcal{C}_l|) \\ \text{s.t.} \quad & \bigcup_{l=1}^K \mathcal{C}_l = \mathcal{X}, \mathcal{C}_l \neq \emptyset, \mathcal{C}_l \cap \mathcal{C}_h = \emptyset, \\ & \mathbf{1}_m^\top \alpha = 1, \alpha \succeq \mathbf{0}, \end{aligned} \quad (3)$$

where $\alpha = [\alpha_1, \dots, \alpha_m]^\top$ is the weight vector, $\gamma > 1$ is introduced to avoid trivial solutions and to encourage an adaptive weighting strategy, $\mathbf{1}_m$ denotes the all-one vector of

length m and the constraints $\mathbf{1}_m^\top \alpha = 1$ and $\alpha \succeq \mathbf{0}$ restrict α to the probability simplex. Further details and discussions on α are provided in Section III-D2.

B. Soft Balancing Against Two-Level Cluster-Size Bias

The unified one-step ensemble framework in problem (3) is designed to mitigate two types of cluster-size bias. First, at the base-clustering representation level, using the CA matrix may cause ambiguous samples to be biased by cluster sizes [39]. To mitigate this bias, we incorporate cluster-size information into the normalized representation detailed in Section III-C. Second, at the discrete one-step optimization level, minimizing super-node similarity in Eq. (1) may also induce cluster-size bias, where candidate clusters with larger sizes tend to dominate the row-wise update, leading to a “rich-get-richer” effect, as further analyzed in Section III-E3. To mitigate this optimization-induced bias, we introduce a balanced regularization [35] that encourages clusters to have flexible sizes without imposing hard equality constraints.

Let $\{|\mathcal{C}_l|\}_{l=1}^K$ denote the cluster sizes, which satisfy $\sum_{l=1}^K |\mathcal{C}_l| = n$ and $|\mathcal{C}_l| \geq 0$. We instantiate the balancing term by using the square-sum of cluster sizes: $f(|\mathcal{C}_l|) = |\mathcal{C}_l|^2$. This formulation enables efficient row-wise optimization in Section III-D, and quadratically penalizes overly large clusters to discourage highly imbalanced partitions.

Theorem 1. *Given $\sum_{l=1}^K |\mathcal{C}_l| = n$ and $|\mathcal{C}_l| \geq 0$, the term $\sum_{l=1}^K |\mathcal{C}_l|^2$ is minimized when $|\mathcal{C}_l| = \dots = |\mathcal{C}_K| = n/K$.*

The proof of Theorem 1 is in the supplementary material.

Remark 1. *To illustrate the effect of soft balancing, we constructed a synthetic dataset with $K = 5$ and 2000 samples per cluster, and generated $m = 5$ base clusterings by k -means with $l_k \in \{3, 4, 5, 6, 7\}$. As shown in Fig. 1, the partition is highly imbalanced without cluster-size bias correction (left), while the normalized representation with balanced regularization yields a more even size distribution (right). Thus, problem (3) prevents degenerate partitions by providing a soft yet effective correction for cluster-size bias, rather than imposing an equal cluster-size prior. Such bias may also arise in imbalanced data, as discussed in Section IV-G.*

C. Problem Reformulation via Normalized Representation

To simplify the problem (3), we introduce an indicator matrix $\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_K] \in \{0, 1\}^{n \times K}$ to denote the consensus clustering result. Each row of $\mathbf{Y}$ contains exactly one entry equal to 1 and all others equal to 0, and $\mathbf{y}_l$ is the indicator vector of the l -th cluster.

Theorem 2. *Let $\mathbf{D}_k = \text{diag}(d_1^k, \dots, d_n^k)$ denote the degree matrix of $\mathbf{W}_k$, where $d_i^k = (\mathbf{D}_k)_{ii} = \sum_{j=1}^n w_{ij}^k$. For any cluster $\mathcal{C}_l$ with indicator vector $\mathbf{y}_l$, the total similarity between $\mathcal{C}_l$ and its complement $\bar{\mathcal{C}}_l$ in the k -th base clustering satisfies $\sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) = \mathbf{y}_l^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{y}_l$, where $\mathbf{y}_l^\top \mathbf{D}_k \mathbf{y}_l = \sum_{i \in \mathcal{C}_l} d_i^k$ and $\mathbf{y}_l^\top \mathbf{W}_k \mathbf{y}_l = \sum_{i \in \mathcal{C}_l} \sum_{j \in \mathcal{C}_l} w_{ij}^k$.*

The proof of Theorem 2 is in the supplementary material.

By Theorem 2, the inter-cluster similarity term over all K clusters can be written as $\sum_{l=1}^K \mathbf{y}_l^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{y}_l = \text{tr}(\mathbf{Y}^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{Y})$. For the balanced regularization, the size of the l -th cluster is given by $|\mathcal{C}_l| = \mathbf{y}_l^\top \mathbf{1}_n$. Thus, the square-sum balancing term can be written as $\sum_{l=1}^K |\mathcal{C}_l|^2 = \sum_{l=1}^K (\mathbf{y}_l^\top \mathbf{1}_n)^2 = \text{tr}(\mathbf{Y}^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{Y})$. Combining the above two identities, problem (3) is rewritten as

$$\begin{aligned} \min_{\mathbf{Y}, \alpha} \sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{Y}) + \lambda \text{tr}(\mathbf{Y}^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{Y}) \\ \text{s.t. } \mathbf{Y} \in \{0, 1\}_{n \times K}, \mathbf{Y} \mathbf{1}_K = \mathbf{1}_n, \mathbf{1}_m^\top \alpha = 1, \alpha \succeq \mathbf{0}. \end{aligned} \quad (4)$$

Given the indicator matrix $\mathbf{A}_k \in \{0, 1\}^{n \times l_k}$ of π_k , directly using the CA matrix $\mathbf{S}_k = \mathbf{A}_k \mathbf{A}_k^\top$ may bias ambiguous samples toward larger clusters and increase imbalance [39]. To remove the large-cluster bias, we incorporate cluster-size information and construct a normalized CA matrix as follows:

$$\mathbf{W}_k = \mathbf{A}_k (\mathbf{A}_k^\top \mathbf{A}_k)^{-1} \mathbf{A}_k^\top = \mathbf{A}_k \Delta_k^{-1} \mathbf{A}_k^\top, \quad (5)$$

where $\Delta_k = \mathbf{A}_k^\top \mathbf{A}_k$ denotes the diagonal cluster-size matrix.

Theorem 3. *The matrix $\mathbf{W}_k$ defined in Eq. (5) is doubly stochastic, i.e., $\mathbf{W}_k \mathbf{1}_n = \mathbf{1}_n$ and $\mathbf{1}_n^\top \mathbf{W}_k = \mathbf{1}_n^\top$. Consequently, the diagonal degree matrix satisfies $\mathbf{D}_k = \mathbf{I}$.*

The proof of Theorem 3 is in the supplementary material.

Let $\mathbf{B}_k = \mathbf{A}_k \Delta_k^{-1/2}$, we have $\mathbf{W}_k = \mathbf{B}_k \mathbf{B}_k^\top$. By Theorem 3, substituting $\mathbf{D}_k = \mathbf{I}$ into problem (4) yields

$$\begin{aligned} \min_{\mathbf{Y}, \alpha} \sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}) + \lambda \text{tr}(\mathbf{Y}^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{Y}) \\ \text{s.t. } \mathbf{Y} \in \{0, 1\}_{n \times K}, \mathbf{Y} \mathbf{1}_K = \mathbf{1}_n, \mathbf{1}_m^\top \alpha = 1, \alpha \succeq \mathbf{0}. \end{aligned} \quad (6)$$

Since $\mathbf{A}_k$ has one nonzero entry per row, $\mathbf{B}_k$ is row-wise one-sparse, which enables the efficient update strategy that follows.

D. Efficient Iterative Optimization

In this section, we propose an efficient iterative optimization strategy to solve problem (6). Specifically, we adopt the block-coordinate descent method [40] to optimize $\mathbf{Y}$ and α by alternately updating one variable while keeping the other fixed.

1) *Update $\mathbf{Y}$:* With α fixed, the $\mathbf{Y}$ -subproblem of problem (6) can be written as

$$\min_{\mathbf{Y} \in \text{Ind}} \sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top \mathbf{Y} - \mathbf{Y}^\top \mathbf{B}_k \mathbf{B}_k^\top \mathbf{Y}) + \lambda \text{tr}(\mathbf{Y}^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{Y}). \quad (7)$$

Since $\mathbf{Y} \in \text{Ind}$ implies $\mathbf{Y} \mathbf{1}_K = \mathbf{1}_n$, we have $\text{tr}(\mathbf{Y}^\top \mathbf{Y}) = n$. Therefore, the term $\sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top \mathbf{Y}) = n \sum_{k=1}^m \alpha_k^\gamma$ is a constant w.r.t. $\mathbf{Y}$. As a result, the $\mathbf{Y}$ -subproblem reduces to

$$\min_{\mathbf{Y} \in \text{Ind}} \sum_{l=1}^K \mathbf{y}_l^\top \left(\lambda \mathbf{1}_n \mathbf{1}_n^\top - \sum_{k=1}^m \alpha_k^\gamma \mathbf{B}_k \mathbf{B}_k^\top \right) \mathbf{y}_l, \quad (8)$$

where $\mathbf{y}_l$ is the l -th column of $\mathbf{Y}$, and $\text{Ind} \triangleq \{\mathbf{Y} \in \{0, 1\}^{n \times K} \mid \mathbf{Y} \mathbf{1}_K = \mathbf{1}_n\}$. Let $\tilde{\mathbf{D}} \triangleq \lambda \mathbf{1}_n \mathbf{1}_n^\top - \sum_{k=1}^m \alpha_k^\gamma \mathbf{B}_k \mathbf{B}_k^\top$, problem (8) is rewritten as

$$\min_{\mathbf{Y} \in \text{Ind}} \mathcal{L}(\mathbf{Y}) = \min_{\mathbf{Y} \in \text{Ind}} \sum_{l=1}^K \mathbf{y}_l^\top \tilde{\mathbf{D}} \mathbf{y}_l, \quad (9)$$

Since $\mathbf{Y}$ is row-independent, we thus use the coordinate descent (CD) algorithm [41] to solve problem (9). Specifically, at each iteration, the optimization over $\mathbf{Y}$ can be decomposed into n simple subproblems, one for each row i ($i = 1, \dots, n$). When updating the i -th row, there are K candidates $\{\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(K)}\}$, where $\mathbf{Y}^{(s)}$ differs from $\mathbf{Y}$ only in the i -th row with $y_{is}^{(s)} = 1$ and $y_{iq}^{(s)} = 0$ for all $q \neq s$. Then the optimal reassignment of $\mathbf{x}_i$ is given by

$$\min_{s \in \{1, \dots, K\}} \mathcal{L}(\mathbf{Y}^{(s)}) = \min_{s \in \{1, \dots, K\}} \sum_{l=1}^K (\mathbf{y}_l^{(s)})^\top \tilde{\mathbf{D}} \mathbf{y}_l^{(s)}. \quad (10)$$

Then, according to the CD algorithm and the nature of the indicator matrix $\mathbf{Y}$, the i -th row of $\mathbf{Y}$ is updated by

$$y_{iq} = \mathbb{I}\left[q = \arg \min_{s \in \{1, \dots, K\}} \mathcal{L}(\mathbf{Y}^{(s)})\right], \quad (11)$$

where $\mathbb{I}[\cdot]$ is the indicator function that equals 1 if the condition holds and 0 otherwise.

However, a direct calculation of $\mathcal{L}(\mathbf{Y}^{(s)})$ is time-consuming, because many terms shared across candidates are repeatedly computed. In particular, when $\mathcal{L}(\mathbf{Y}^{(s)})$ is calculated separately for each candidate, the dominant cost comes from $\mathbf{B}_k^\top \mathbf{y}_l^{(s)}$ for $k = 1, \dots, m$ and $l = 1, \dots, K$, where $\mathbf{B}_k$ has exactly one nonzero per row, leading to $\mathcal{O}(nKm)$ per candidate. When all n rows are updated by checking K candidates, the overall complexity is $\mathcal{O}(n^2 K^2 m)$ per iteration, which becomes inefficient when n is large.

Acceleration Strategy 1: incremental computation

To eliminate the repeated computation of shared terms across candidate clusters in the i -th row update, we introduce an auxiliary matrix $\mathbf{Y}^{(0)}$, which has the same definition as $\mathbf{Y}^{(s)}$ except that all entries in the i -th row are set to zero. Then the row update in Eq. (11) is equivalent to selecting the cluster index that minimizes the objective difference:

$$\begin{aligned} \min_{s \in \{1, \dots, K\}} \varphi(s) &= \min_{s \in \{1, \dots, K\}} \left(\mathcal{L}(\mathbf{Y}^{(s)}) - \mathcal{L}(\mathbf{Y}^{(0)}) \right) \\ &\Leftrightarrow \min_{s \in \{1, \dots, K\}} \sum_{l=1}^K \left((\mathbf{y}_l^{(s)})^\top \tilde{\mathbf{D}} \mathbf{y}_l^{(s)} - (\mathbf{y}_l^{(0)})^\top \tilde{\mathbf{D}} \mathbf{y}_l^{(0)} \right) \\ &\Leftrightarrow \min_{s \in \{1, \dots, K\}} \left((\mathbf{y}_s^{(s)})^\top \tilde{\mathbf{D}} \mathbf{y}_s^{(s)} - (\mathbf{y}_s^{(0)})^\top \tilde{\mathbf{D}} \mathbf{y}_s^{(0)} \right) \\ &\Leftrightarrow \min_{s \in \{1, \dots, K\}} \left(\lambda \left((\mathbf{y}_s^{(s)})^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{y}_s^{(s)} - (\mathbf{y}_s^{(0)})^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{y}_s^{(0)} \right) \right. \\ &\quad \left. - \sum_{k=1}^m \alpha_k^\gamma \left((\mathbf{y}_s^{(s)})^\top \mathbf{B}_k \mathbf{B}_k^\top \mathbf{y}_s^{(s)} - (\mathbf{y}_s^{(0)})^\top \mathbf{B}_k \mathbf{B}_k^\top \mathbf{y}_s^{(0)} \right) \right). \end{aligned} \quad (12)$$

Acceleration Strategy 2: storage reduction

However, it is worth noting that, when updating the i -th row of $\mathbf{Y}$, directly minimizing $\varphi(s)$ in problem (12) would require

storing $K+1$ variables $\{\mathbf{Y}^{(s)}\}_{s=1}^K$ and $\mathbf{Y}^{(0)}$. Therefore, before each iteration, we assume the current indicator matrix is $\mathbf{Y}$, and use the current variable $\mathbf{Y}$ to replace these variables to reduce the space storage. Let p ($p = 1, \dots, K$) record the location of the entry 1 in the i -th row, and δ_i be the one-hot vector whose i -th entry is 1 and all other entries are 0. Then the corresponding columns satisfy

$$\mathbf{y}_s^{(s)} = \begin{cases} \mathbf{y}_s, & \text{if } s = p, \\ \mathbf{y}_s + \delta_i, & \text{otherwise,} \end{cases} \quad (13)$$

$$\mathbf{y}_s^{(0)} = \begin{cases} \mathbf{y}_s - \delta_i, & \text{if } s = p, \\ \mathbf{y}_s, & \text{otherwise.} \end{cases} \quad (14)$$

Based on Eqs. (13) and (14), the intermediate variables needed to compute $\varphi(s)$ have different closed forms for the two cases $s = p$ and $s \neq p$. For clarity, we summarize these expressions in Table II, where $(\mathbf{b}_k)_i$ denotes the i -th row of $\mathbf{B}_k$.

TABLE II
EXPRESSIONS UNDER DIFFERENT CONDITIONS.

Expression	$s = p$	otherwise
$(\mathbf{y}_s^{(s)})^\top \mathbf{1}_n$	$\mathbf{y}_s^\top \mathbf{1}_n$	$\mathbf{y}_s^\top \mathbf{1}_n + 1$
$(\mathbf{y}_s^{(0)})^\top \mathbf{1}_n$	$\mathbf{y}_s^\top \mathbf{1}_n - 1$	$\mathbf{y}_s^\top \mathbf{1}_n$
$(\mathbf{y}_s^{(s)})^\top \mathbf{B}_k$	$\mathbf{y}_s^\top \mathbf{B}_k$	$\mathbf{y}_s^\top \mathbf{B}_k + (\mathbf{b}_k)_i$
$(\mathbf{y}_s^{(0)})^\top \mathbf{B}_k$	$\mathbf{y}_s^\top \mathbf{B}_k - (\mathbf{b}_k)_i$	$\mathbf{y}_s^\top \mathbf{B}_k$

Substituting the results in Table II into problem (12) yields

$$\varphi(s) = \begin{cases} \lambda (2\mathbf{y}_s^\top \mathbf{1}_n - 1) - 2 \sum_{k=1}^m \alpha_k^\gamma \mathbf{y}_s^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top \\ \quad + \sum_{k=1}^m \alpha_k^\gamma (\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top, & \text{if } s = p, \\ \lambda (2\mathbf{y}_s^\top \mathbf{1}_n + 1) - 2 \sum_{k=1}^m \alpha_k^\gamma \mathbf{y}_s^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top \\ \quad - \sum_{k=1}^m \alpha_k^\gamma (\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top, & \text{otherwise.} \end{cases} \quad (15)$$

Accordingly, the update of the i -th row is obtained by selecting the index that minimizes $\varphi(s)$:

$$y_{iq} = \mathbb{I}\left[q = \arg \min_{s \in \{1, \dots, K\}} \varphi(s)\right]. \quad (16)$$

To further accelerate the computation of $\varphi(s)$ in Eq. (15), we analyze and propose an efficient strategy to reduce time complexity. Although $\mathbf{Y}$ can be updated by Eq. (16), we have to calculate $\varphi(s)$ ($s = 1, \dots, K$) with time complexity $\mathcal{O}(nKm)$, where, computing $\mathbf{y}_s^\top \mathbf{B}_k$ and $\mathbf{y}_s^\top \mathbf{1}_n$ for $s = 1, \dots, K$ and $k = 1, \dots, m$ needs $\mathcal{O}(nKm)$ and $\mathcal{O}(nK)$, respectively, and computing $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ ($i = 1, 2, \dots, n$) for all k needs $\mathcal{O}(nm)$. To reduce the time complexity from $\mathcal{O}(nKm)$ to $\mathcal{O}(Km)$, we compute and store $\mathbf{y}_s^\top \mathbf{B}_k$, $\mathbf{y}_s^\top \mathbf{1}_n$ and $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ once in advance. Then, to compute $\arg \min_{s \in \{1, \dots, K\}} \varphi(s)$, we only need to evaluate $\mathbf{y}_s^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top$ for K times, whose time complexity is $\mathcal{O}(Km)$.

Acceleration Strategy 3: variable update

Furthermore, $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ is computed in advance and therefore requires no updates. For the variables $\mathbf{y}_s^\top \mathbf{B}_k$ and $\mathbf{y}_s^\top \mathbf{1}_n$, there are two cases. Suppose $\mathbf{x}_i$ is reassigned from cluster p to q . If $q = p$, all intermediate variables remain unchanged; otherwise, only the variables associated with clusters p and q need to be updated by simple additive rules:

$$\mathbf{y}_p^\top \mathbf{B}_k \leftarrow (\mathbf{y}_p - \delta_i)^\top \mathbf{B}_k = \mathbf{y}_p^\top \mathbf{B}_k - (\mathbf{b}_k)_i, \quad (17)$$

$$\mathbf{y}_q^\top \mathbf{B}_k \leftarrow (\mathbf{y}_q + \delta_i)^\top \mathbf{B}_k = \mathbf{y}_q^\top \mathbf{B}_k + (\mathbf{b}_k)_i, \quad (18)$$

$$\mathbf{y}_p^\top \mathbf{1}_n \leftarrow (\mathbf{y}_p - \delta_i)^\top \mathbf{1}_n = \mathbf{y}_p^\top \mathbf{1}_n - 1, \quad (19)$$

$$\mathbf{y}_q^\top \mathbf{1}_n \leftarrow (\mathbf{y}_q + \delta_i)^\top \mathbf{1}_n = \mathbf{y}_q^\top \mathbf{1}_n + 1. \quad (20)$$

Among them, the entries on the right side of the arrow have been stored in advance and do not need to be recalculated.

2) *Update α* : With $\mathbf{Y}$ fixed, the α -subproblem of problem (6) can be formulated as

$$\min_{\mathbf{1}_m^\top \alpha = 1, \alpha \geq 0} \sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}). \quad (21)$$

Theorem 4. For $\gamma > 1$, the optimal solution to problem (21) admits the following closed-form update:

$$\alpha_k = \frac{(\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}))^{-1/(\gamma-1)}}{\sum_{k=1}^m (\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}))^{-1/(\gamma-1)}}. \quad (22)$$

The proof of Theorem 4 is in the supplementary material.

Remark 2. Eq. (22) gives an adaptive weighting strategy, where base clusterings more consistent with $\mathbf{Y}$ receive larger weights, and vice versa.

The procedure of OLEC-SB is summarized in Algorithm 1.

3) *Complexity Analysis of Algorithm 1*: In this subsection, we will analyze the time and space complexity of Algorithm 1.

Construct $\{\mathbf{B}_k\}_{k=1}^m$. Since $\mathbf{A}_k$ has exactly one nonzero per row, computing cluster sizes Δ_k and scaling nonzero entries to obtain $\mathbf{B}_k$ both cost $\mathcal{O}(n)$ for each k . Hence, constructing all $\{\mathbf{B}_k\}_{k=1}^m$ costs $\mathcal{O}(nm)$.

Update $\mathbf{Y}$. For the variables cached in Step 3, computing $\mathbf{y}_s^\top \mathbf{B}_k$ and $\mathbf{y}_s^\top \mathbf{1}_n$ needs nm and n additions, respectively, while computing $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ needs nm multiplications. When updating the i -th row, computing $\varphi(s)$ ($s = 1, 2, \dots, K$) by Eq. (15) in Step 8 only needs to evaluate $\mathbf{y}_s^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top$, which requires Km multiplications. The time complexity of updating the i -th row of $\mathbf{Y}$ by Eq. (16) in Step 9 can be ignored compared to this step. If $p \neq q$, updating $\mathbf{y}_p^\top \mathbf{B}_k$ and $\mathbf{y}_q^\top \mathbf{B}_k$ by Eqs. (17)-(18) in Step 11 needs $2m$ additions, and updating $\mathbf{y}_p^\top \mathbf{1}_n$ and $\mathbf{y}_q^\top \mathbf{1}_n$ by Eqs. (19)-(20) in Step 12 needs 2 additions. Since there are n loops from Step 7 to Step 14, one inner CD sweep needs $nKm + nm$ multiplications and $nm + n + r(2m + 2)$ additions, where $0 \leq r \leq n$ is the number of changed assignments. Since operating multiplication requires more time than operating addition, we only consider the multiplication of our method. Assuming the algorithm converges after t_Y iterations, its time complexity can be summarized as $\mathcal{O}(nKmt_Y)$.

Algorithm 1 Algorithm to Solve Problem (4)

Input: Binary matrices $\{\mathbf{A}_k\}_{k=1}^m$, number of clusters K

Output: Indicator matrix $\mathbf{Y}$

```
1: Initialize  $\mathbf{Y} \in \text{Ind}$  and  $\alpha_k = 1/m$ , for  $k = 1, \dots, m$ ;  
2: Calculate  $\mathbf{B}_k = \mathbf{A}_k \Delta_k^{-1/2}$ , for  $k = 1, \dots, m$ ;  
3: Compute and store  $\mathbf{y}_s^\top \mathbf{B}_k$ ,  $\mathbf{y}_s^\top \mathbf{1}_n$  and  $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ , for  
    $s = 1, \dots, K$ ,  $k = 1, \dots, m$ , and  $i = 1, \dots, n$ ;  
4: while not converge do  
5:   // Update  $\mathbf{Y}$  with fixed  $\alpha$   
6:   while not converge do  
7:     for  $i = 1$  to  $n$  do  
8:       Calculate  $\varphi(s)$  ( $s = 1, 2, \dots, K$ ) by Eq. (15);  
9:       Update the  $i$ th row of  $\mathbf{Y}$  by Eq. (16);  
10:      if  $p \neq q$  then  
11:        Update  $\mathbf{y}_p^\top \mathbf{B}_k$  and  $\mathbf{y}_q^\top \mathbf{B}_k$  by Eqs. (17) and (18);  
12:        Update  $\mathbf{y}_p^\top \mathbf{1}_n$  and  $\mathbf{y}_q^\top \mathbf{1}_n$  by Eqs. (19) and (20);  
13:      end if  
14:    end for  
15:  end while  
16:  // Update  $\alpha$  with fixed  $\mathbf{Y}$   
17:  for  $k = 1$  to  $m$  do  
18:    Update  $\alpha_k$  by Eq. (22);  
19:  end for  
20: end while
```

Update α . Computing the objective value of each base clustering mainly requires $\mathcal{O}(n)$ for each k and $\mathcal{O}(nm)$ in total. Then, updating α_k by normalization costs $\mathcal{O}(m)$. Thus, the time complexity for updating α is $\mathcal{O}(nm)$.

Let t denote the number of outer iterations in Algorithm 1. The overall time complexity of OLEC-SB is $\mathcal{O}(nm + t(nKmt_Y + nm))$, which is dominated by $\mathcal{O}(tnKmt_Y)$ since $K, m, t, t_Y \ll n$. For bounded K, m, t, t_Y independent of n , OLEC-SB scales linearly with n . The empirical values of t and t_Y are reported in Section IV-F. Table IV summarizes the dominant time complexities of all compared methods, and the space complexity is provided in the supplementary material.

E. Theoretical Analysis

1) *Convergence Analysis:* In each iteration, Algorithm 1 monotonically decreases the objective value of problem (6) until convergence. The proof is in the supplementary material.

2) *Averaged Normalized-Cut Interpretation:* The objective in problem (6) can be interpreted as a variant of the averaged normalized-cut (Ncut). Given the similarity matrix $\mathbf{W}_k$ of the k -th base clustering, the averaged Ncut objective over m base clusterings can be written as

$$\min_{\mathbf{F}^\top \mathbf{F} = \mathbf{I}} \frac{1}{m} \text{tr} \left(\mathbf{F}^\top \left(\mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2} \right) \mathbf{F} \right), \quad (23)$$

where $\mathbf{W} = \sum_{k=1}^m \mathbf{W}_k$ is the similarity matrix over all base clusterings, $\mathbf{D} = \text{diag}(\mathbf{W} \mathbf{1}_n)$ is the degree matrix, and $\mathbf{F} = \mathbf{D}^{1/2} \mathbf{Y} (\mathbf{Y}^\top \mathbf{D} \mathbf{Y})^{-1/2}$ is the spectral embedding.

Remark 3. Averaged Ncut is related to OLEC-SB in two aspects. First, the normalization by cluster volume $\mathbf{y}_i^\top \mathbf{D} \mathbf{y}_i$ reduces representation-level large-cluster bias, aligning with the normalized bipartite representation $\mathbf{B}_k = \mathbf{A}_k \Delta_k^{-1/2}$ in OLEC-SB. Second, the factor $1/m$ assigns equal weights to all base clusterings, equivalent to fixing $\alpha_k = 1/m$ in OLEC-SB. Thus, averaged Ncut can be viewed as a uniform-weight and representation-level bias-correction form of OLEC-SB.

3) Optimization-Induced Cluster-Size Bias Correction:

Theorem 5. Let $\mathcal{C}_s^{-i}$ denote the s -th cluster of $\mathbf{Y}^{(0)}$ after temporarily removing $\mathbf{x}_i$. For each candidate cluster s , let $|\mathcal{C}_s^{-i}| = (\mathbf{y}_s^{(0)})^\top \mathbf{1}_n$ be the size of $\mathcal{C}_s^{-i}$, $\bar{a}_{is} = 1/|\mathcal{C}_s^{-i}| \sum_{k=1}^m \alpha_k^\gamma \sum_{j \in \mathcal{C}_s^{-i}} (\mathbf{W}_k)_{ji}$ denote the average similarity between $\mathbf{x}_i$ and $\mathcal{C}_s^{-i}$. Then, the row-wise update of $\mathbf{x}_i$ is equivalent to $q = \arg \max_{s \in \{1, \dots, K\}} (|\mathcal{C}_s^{-i}| \bar{a}_{is} - \lambda |\mathcal{C}_s^{-i}|)$.

The proof of Theorem 5 is in the supplementary material.

Remark 4. The row-wise update of $\mathbf{x}_i$ is determined by a total weighted similarity term and a cluster-size correction term. When $\lambda = 0$, the assignment depends on $|\mathcal{C}_s^{-i}| \bar{a}_{is}$. Thus, even if two clusters have comparable average similarity to $\mathbf{x}_i$, the larger one is more likely to be selected, yielding a clear "rich-get-richer" effect. The term $-\lambda |\mathcal{C}_s^{-i}|$ offsets this optimization-induced large-cluster bias, consistent with Section III-B.

IV. EXPERIMENT

A. Experimental Setup

We conducted experiments on 10 datasets with different scales and cluster-size distributions, including six medium-scale datasets D1-D6 and four large-scale datasets D7-D10, as summarized in Table III. Among them, D2, D3, and D7 are balanced, while the others are imbalanced. We compared OLEC-SB with 14 baseline methods, including scalable ensemble clustering methods (SEC [31], ECPCS-MC [17], U-SENC [13], FSEC [22], and YACHT [21]), balanced ensemble clustering methods (FCE [10] and TBAWEC [37]), and other representative ensemble clustering methods (PTGP [12], LWEA [6], LWGP [6], ECPCS-HC [17], ECCMS [7], SDGCA [5], and GPEC [8]). Publicly available code and recommended parameter settings were used for all baseline methods. Experiments were conducted in MATLAB R2024a on a shared Windows server with two Intel Xeon Gold 6248R CPUs at 3.00 GHz and 768 GB RAM, using a fixed random seed `rng(2026)`. Table IV summarizes the dominant time complexity terms of all methods and indicates whether each method is scalable, one-step, and balance-aware. Further details on the datasets and baseline methods are provided in the supplementary material. For each dataset, we first generated 100 candidate base clusterings by running k -means with $l_k \in [2, \sqrt{n}]$. We then randomly selected 20 base clusterings, repeated this process 10 times, and calculated the mean and standard deviation. We used accuracy (ACC) [42], adjusted rand index (ARI) [7], F-score [5], and NE [10] for evaluation, where larger ACC, ARI and F-score indicate better clustering quality, and larger NE indicates better balance performance.

TABLE III
DETAILS OF DIFFERENT DATASETS.

No.	Dataset	#Instance	#Feature	#Class
D1	Maternal Health Risk	1014	6	3
D2	CNAE-9	1080	857	9
D3	Phishing Websites	2456	30	2
D4	PSDAS	4424	36	3
D5	TUANDROMD	4464	241	2
D6	Gender Gap	4746	18	3
D7	COIL100	7200	1024	100
D8	Crowdsourced Mapping	10845	29	6
D9	HTRU2	17898	8	2
D10	Steel Energy	35040	9	3

TABLE IV
COMPARISON OF TIME COMPLEXITY OF ALL METHODS.

Method	Time complexity	Scalable	One-Step	Balance
PTGP [12]	$\mathcal{O}(n^{2.5})$			
SEC [31]	$\mathcal{O}(tnKmd)$	✓		
LWEA [6]	$\mathcal{O}(n^2 \sum_{k=1}^m l_k)$			
LWGP [6]	$\mathcal{O}(n^2 \sum_{k=1}^m l_k)$			
ECPCS-HC [17]	$\mathcal{O}(n^2 \sum_{k=1}^m l_k)$			
ECPCS-MC [17]	$\mathcal{O}(tnKmd)$	✓		
U-SENC [13]	$\mathcal{O}(nmp^{1/2}d)$	✓		
ECCMS [7]	$\mathcal{O}(tn^{2.37})$			
FSEC [22]	$\mathcal{O}(nKp)$	✓		
YACHT [21]	$\mathcal{O}(ts'np)$	✓		
SDGCA [5]	$\mathcal{O}(tn^{2.37})$			
GPEC [8]	$\mathcal{O}(tn^2(m+K))$			
FCE [10]	$\mathcal{O}(tn^2K^2)$		✓	✓
TBAWEC [37]	$\mathcal{O}(tn^2K)$		✓	✓
OLEC-SB (Ours)	$\mathcal{O}(tnKmt_Y)$	✓	✓	✓

The time complexities listed in this table are dominant terms for comparison. p denotes the number of anchors, and s' is a unique parameter in YACHT.

B. Clustering Performance

Table V presents the running time and ACC results on different datasets, and Table VI shows the NE results. The ARI and F-score results are provided in the supplementary material. Based on these results, we make the following observations: (i) OLEC-SB achieves superior performance on datasets of different scales, improving over the second-best method by 13.2%, 18.9%, and 10.4% on average in ACC, ARI, and F-score, respectively. (ii) On the balanced datasets D2 and D3, OLEC-SB and TBAWEC achieve large NE values in Table VI, matching the balanced cluster-size distributions of these datasets and verifying the role of soft balancing. Thus, they obtain the best and second-best results on all three metrics, respectively. However, TBAWEC cannot be applied to large-scale datasets D7-D10 due to its high computational cost. (iii) On the imbalanced datasets D1, D4 and D5, OLEC-SB does not impose an equal cluster-size prior; thus, its NE values are not excessively large and better reflect the intrinsic imbalanced distributions. As a result, OLEC-SB still achieves superior clustering performance. In contrast, TBAWEC tends to keep larger NE values, which overemphasizes balance and degrades clustering performance. (iv) OLEC-SB achieves practical efficiency on both medium-scale and large-scale datasets. Compared with the balanced

methods FCE and TBAWEC, it achieves about $2.9\times$ speedup over FCE on average and about $1600\times$ speedup over TBAWEC on D1-D6. (v) Although some scalable methods run faster due to simpler non-iterative procedures, they often overlook mismatch issues and cluster-size bias. In contrast, the efficient iterative optimization strategy of OLEC-SB integrates soft balancing and adaptive weighting within a mismatch-free one-step ensemble framework, achieving better performance while remaining efficient. (vi) Overall, existing scalable methods generally do not outperform the state-of-the-art (SOTA) methods on most datasets; however, SOTA methods often scale poorly and may fail to return results on large-scale datasets. To provide a more intuitive view, we further visualize the running time of all compared methods on each dataset in Figure 3.

C. Empirical Impact of Soft Balancing

We evaluated the balancing effect of OLEC-SB on the balanced dataset D7. Figure 4 shows the cluster-size distributions of different methods, with all 15 methods provided in the supplementary material. Most methods generate highly imbalanced partitions, indicating the difficulty of achieving balanced clustering. In contrast, FCE and OLEC-SB produce much more even cluster-size distributions with larger NE values. Consistently, both methods also achieve large NE values on the balanced datasets D2 and D3. However, as shown in Table V, FCE brings only limited improvement in ACC, ARI, and F-score. This may be attributed to its orthogonal alignment and continuous relaxation before discretization, which can introduce a relaxation-discretization mismatch.

D. Ablation Study

To verify the effectiveness of the core components in OLEC-SB, *i.e.*, the normalized representation, balanced regularization, and adaptive weighting, we conducted an ablation study on three variants: w/o normalization, w/o balancing (setting $\lambda = 0$), and w/o weighting (fixing $\alpha_k = 1/m$). Table VII summarizes the results in terms of ACC, ARI and F-score. We observe that removing any component leads to performance degradation. Specifically, without normalization, the representation-level correction for cluster-size bias in CA matrices is weakened, resulting in average drops of 17.5% in ACC, 27.1% in ARI, and 13.7% in F-score. Removing balanced regularization tends to produce more imbalanced assignments, leading to noticeable drops on the balanced datasets (D2, D3 and D7) across all three metrics, with average drops of 15.6% in ACC, 37.1% in ARI, and 7.5% in F-score. Without adaptive weighting, all base clusterings contribute equally and inconsistent ones cannot be suppressed, resulting in a more substantial overall performance decline, with average drops of 25.1% in ACC, 31.7% in ARI, and 22.5% in F-score. These findings validate the effectiveness of integrating the normalized representation, balanced regularization, and adaptive weighting into the unified one-step ensemble framework.

E. Ensemble Size Analysis

Figure 5 shows the performance of all methods with different numbers of base clusterings. On most datasets, the

TABLE V
PERFORMANCE COMPARISON ON DIFFERENT DATASETS IN TERMS OF RUNNING TIME (SEC.) AND ACC (MEAN $\pm$ STD, %). BEST RESULTS ARE HIGHLIGHTED IN **BOLD**, AND SECOND-BEST RESULTS ARE MARKED WITH AN UNDERLINE.

Method	Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	Avg.
PTGP [12]	time↓	0.04	0.20	0.65	3.06	0.36	1.95	3.72	178.95	1601.57	51.33	184.18
	ACC↑	49.5 $\pm$ 5.2	44.3 $\pm$ 3.8	51.2 $\pm$ 1.0	42.8 $\pm$ 4.1	76.5 $\pm$ 6.8	55.8 $\pm$ 4.5	26.7 $\pm$ 1.1	43.0 $\pm$ 3.7	89.1 $\pm$ 0.5	49.8 $\pm$ 2.0	52.9 $\pm$ 3.3
SEC [31]	time↓	0.00	0.01	0.01	0.03	0.01	0.03	0.79	0.16	0.13	0.47	0.16
	ACC↑	52.4 $\pm$ 2.2	41.6 $\pm$ 4.7	54.3 $\pm$ 6.3	49.3 $\pm$ 4.2	72.4 $\pm$ 10.7	59.2 $\pm$ 6.2	41.6 $\pm$ 1.2	<u>50.5</u> $\pm$ 11.0	91.5 $\pm$ 6.2	49.9 $\pm$ 5.8	56.3 $\pm$ 5.9
LWEA [6]	time↓	0.25	0.30	1.14	3.31	3.61	3.28	5.51	15.16	53.73	152.54	23.88
	ACC↑	55.6 $\pm$ 1.0	47.1 $\pm$ 2.9	51.9 $\pm$ 0.1	45.5 $\pm$ 3.7	79.6 $\pm$ 3.8	70.5 $\pm$ 2.1	35.4 $\pm$ 1.0	46.5 $\pm$ 6.1	<u>93.8</u> $\pm$ 3.1	49.6 $\pm$ 2.1	57.6 $\pm$ 2.6
LWGP [6]	time↓	0.09	0.16	0.19	0.33	0.28	0.36	1.57	0.88	1.38	3.17	0.84
	ACC↑	56.6 $\pm$ 4.6	49.1 $\pm$ 2.1	52.1 $\pm$ 0.1	42.9 $\pm$ 2.4	76.7 $\pm$ 3.7	70.8 $\pm$ 0.3	44.0 $\pm$ 1.3	47.9 $\pm$ 4.2	92.1 $\pm$ 0.2	50.5 $\pm$ 0.1	58.3 $\pm$ 1.9
ECPCS-HC [17]	time↓	0.22	0.70	1.19	3.62	3.72	14.21	17.69	45.00	67.77	262.42	41.66
	ACC↑	56.7 $\pm$ 0.9	40.8 $\pm$ 3.5	51.8 $\pm$ 0.6	44.9 $\pm$ 3.0	75.6 $\pm$ 4.8	71.1 $\pm$ 0.4	32.7 $\pm$ 1.5	45.8 $\pm$ 0.7	92.6 $\pm$ 2.4	44.5 $\pm$ 4.3	55.7 $\pm$ 2.2
ECPCS-MC [17]	time↓	0.09	0.17	0.32	0.73	0.73	1.21	3.87	5.08	4.59	12.55	2.93
	ACC↑	<u>57.6</u> $\pm$ 4.6	40.9 $\pm$ 3.1	51.5 $\pm$ 0.9	45.1 $\pm$ 3.3	78.1 $\pm$ 0.5	<u>70.9</u> $\pm$ 0.0	38.7 $\pm$ 1.2	44.3 $\pm$ 3.4	91.1 $\pm$ 0.5	40.9 $\pm$ 6.5	55.9 $\pm$ 2.4
U-SENC [13]	time↓	0.02	0.04	0.06	0.13	0.10	0.13	0.85	0.47	0.85	1.71	0.43
	ACC↑	51.9 $\pm$ 6.9	47.9 $\pm$ 2.7	51.7 $\pm$ 1.1	44.9 $\pm$ 2.8	74.2 $\pm$ 5.8	66.5 $\pm$ 5.4	44.4 $\pm$ 1.3	44.3 $\pm$ 3.4	91.2 $\pm$ 0.3	46.5 $\pm$ 6.6	56.3 $\pm$ 3.6
ECCMS [7]	time↓	9.35	11.99	67.75	340.66	424.44	401.62	1422.21	4245.39	T/O	T/O	–
	ACC↑	57.2 $\pm$ 1.7	45.9 $\pm$ 3.6	51.8 $\pm$ 0.6	46.2 $\pm$ 4.4	79.2 $\pm$ 4.0	69.5 $\pm$ 2.1	32.9 $\pm$ 1.5	48.1 $\pm$ 1.9	T/O	T/O	–
FSEC [22]	time↓	0.27	0.50	0.72	13.94	13.93	1.49	3.05	5.43	22.54	21.20	8.31
	ACC↑	48.0 $\pm$ 6.5	53.9 $\pm$ 3.9	55.5 $\pm$ 3.5	<u>49.8</u> $\pm$ 1.6	<u>94.4</u> $\pm$ 0.7	62.5 $\pm$ 6.6	41.1 $\pm$ 1.5	44.3 $\pm$ 7.0	86.4 $\pm$ 2.1	48.5 $\pm$ 4.6	<u>58.4</u> $\pm$ 3.8
YACHT [21]	time↓	1.56	1.14	3.72	20.18	12.83	10.18	23.18	94.48	81.06	178.68	36.70
	ACC↑	51.1 $\pm$ 3.6	53.7 $\pm$ 4.8	74.8 $\pm$ 10.2	46.0 $\pm$ 5.0	71.4 $\pm$ 7.4	61.9 $\pm$ 6.8	<u>47.5</u> $\pm$ 1.7	48.3 $\pm$ 7.8	58.1 $\pm$ 6.0	<u>51.8</u> $\pm$ 6.5	56.5 $\pm$ 6.0
SDGCA [5]	time↓	27.87	25.13	121.11	601.67	1283.63	1250.41	4525.16	T/O	T/O	OOM	–
	ACC↑	56.9 $\pm$ 4.9	46.6 $\pm$ 4.1	52.0 $\pm$ 0.1	47.1 $\pm$ 4.6	78.0 $\pm$ 0.2	69.9 $\pm$ 1.6	33.3 $\pm$ 1.4	T/O	T/O	OOM	–
GPEC [8]	time↓	62.97	100.08	231.57	777.87	942.59	861.61	3999.35	T/O	T/O	OOM	–
	ACC↑	48.0 $\pm$ 2.9	50.4 $\pm$ 4.0	78.8 $\pm$ 4.3	43.4 $\pm$ 2.3	56.4 $\pm$ 3.0	58.2 $\pm$ 5.1	47.3 $\pm$ 1.2	T/O	T/O	OOM	–
FCE [10]	time↓	1.27	1.34	3.60	10.88	7.93	9.92	242.65	55.48	90.79	393.27	81.71
	ACC↑	43.2 $\pm$ 3.4	53.3 $\pm$ 4.5	54.6 $\pm$ 3.1	38.8 $\pm$ 2.2	51.2 $\pm$ 0.9	48.8 $\pm$ 5.0	44.2 $\pm$ 1.6	24.8 $\pm$ 1.1	50.1 $\pm$ 0.0	38.3 $\pm$ 2.4	44.7 $\pm$ 2.4
TBAWEC [37]	time↓	30.47	66.13	249.51	2461.23	1378.39	3077.75	T/O	T/O	T/O	T/O	–
	ACC↑	49.7 $\pm$ 2.9	<u>55.7</u> $\pm$ 4.8	83.3 $\pm$ 17.2	44.6 $\pm$ 3.6	67.5 $\pm$ 6.2	54.6 $\pm$ 1.7	T/O	T/O	T/O	T/O	–
OLEC-SB (Ours)	time↓	0.07	0.33	0.86	1.16	1.09	1.00	17.32	6.28	12.86	244.77	28.57
	ACC↑	60.3 $\pm$ 3.0	61.5 $\pm$ 3.4	88.6 $\pm$ 4.7	56.1 $\pm$ 2.7	95.5 $\pm$ 0.6	69.0 $\pm$ 2.9	52.0 $\pm$ 1.5	72.2 $\pm$ 2.5	97.1 $\pm$ 0.8	64.1 $\pm$ 0.9	71.6 $\pm$ 2.3

T/O indicates excessive runtime (mean execution time over 10 runs $>$ 5000 s), and OOM indicates insufficient memory. Light-gray rows denote the balance-aware baselines FCE and TBAWEC, while the dark-gray row denotes the proposed OLEC-SB.

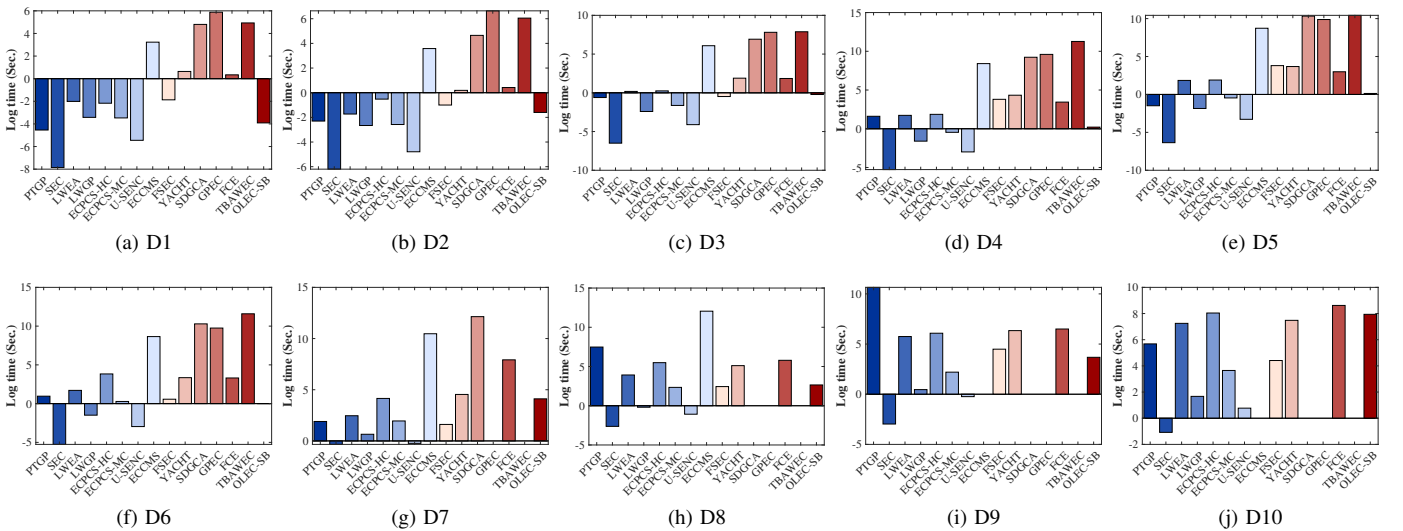


Fig. 3. Log-transformed absolute runtime of different methods on datasets D1–D10, where negative values indicate runtimes below one second. Missing bars denote T/O for ECCMS on D9–D10, GPEC and SDGCA on D8–D9, and TBAWEC on D7–D10, and OOM for GPEC and SDGCA on D10.

TABLE VI
PERFORMANCE COMPARISON ON DIFFERENT DATASETS IN TERMS OF NE (MEAN $\pm$ STD, %).

Method	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
PTGP [12]	93.2 $\pm$ 3.4	77.0 $\pm$ 4.7	74.4 $\pm$ 9.5	93.5 $\pm$ 3.8	39.5 $\pm$ 33.2	75.3 $\pm$ 16.6	78.4 $\pm$ 1.1	96.6 $\pm$ 1.9	67.4 $\pm$ 1.6	96.4 $\pm$ 1.9
SEC [31]	68.5 $\pm$ 11.7	77.8 $\pm$ 7.7	61.3 $\pm$ 16.6	50.1 $\pm$ 13.0	53.6 $\pm$ 12.1	60.2 $\pm$ 22.6	94.4 $\pm$ 0.9	69.7 $\pm$ 15.9	27.6 $\pm$ 14.0	33.5 $\pm$ 16.8
LWEA [6]	79.2 $\pm$ 6.1	73.1 $\pm$ 7.5	52.6 $\pm$ 2.9	90.9 $\pm$ 7.3	23.0 $\pm$ 16.7	83.8 $\pm$ 4.9	84.9 $\pm$ 1.0	86.2 $\pm$ 8.4	49.2 $\pm$ 12.3	94.8 $\pm$ 3.2
LWGP [6]	86.6 $\pm$ 4.3	81.4 $\pm$ 5.1	51.4 $\pm$ 0.3	89.2 $\pm$ 5.7	15.3 $\pm$ 0.0	83.4 $\pm$ 0.1	92.7 $\pm$ 0.4	91.9 $\pm$ 2.4	58.0 $\pm$ 2.7	96.0 $\pm$ 0.1
ECPCS-HC [17]	77.6 $\pm$ 2.1	52.7 $\pm$ 5.6	57.0 $\pm$ 12.9	90.5 $\pm$ 7.0	23.7 $\pm$ 17.6	82.4 $\pm$ 2.4	81.3 $\pm$ 1.5	92.0 $\pm$ 1.5	55.9 $\pm$ 8.4	79.0 $\pm$ 12.5
ECPCS-MC [17]	79.7 $\pm$ 11.8	52.8 $\pm$ 4.9	71.0 $\pm$ 16.8	92.9 $\pm$ 4.0	14.3 $\pm$ 3.4	83.8 $\pm$ 0.1	87.6 $\pm$ 0.9	94.3 $\pm$ 3.4	61.8 $\pm$ 1.3	78.6 $\pm$ 12.0
U-SENC [13]	87.8 $\pm$ 3.3	82.5 $\pm$ 2.6	65.5 $\pm$ 15.1	95.0 $\pm$ 4.5	43.0 $\pm$ 37.9	89.9 $\pm$ 7.5	93.6 $\pm$ 0.5	95.5 $\pm$ 1.9	61.5 $\pm$ 1.0	88.6 $\pm$ 12.0
ECCMS [7]	82.2 $\pm$ 4.6	72.3 $\pm$ 7.2	53.8 $\pm$ 8.9	90.4 $\pm$ 8.9	18.9 $\pm$ 12.3	84.1 $\pm$ 2.0	79.0 $\pm$ 2.1	89.4 $\pm$ 5.1	T/O	T/O
FSEC [22]	85.5 $\pm$ 6.5	96.6 $\pm$ 1.9	83.1 $\pm$ 15.1	72.2 $\pm$ 5.4	69.4 $\pm$ 1.5	91.3 $\pm$ 5.5	89.1 $\pm$ 1.2	93.5 $\pm$ 4.5	73.4 $\pm$ 4.2	78.6 $\pm$ 15.9
YACHT [21]	94.8 $\pm$ 3.1	93.5 $\pm$ 1.8	92.3 $\pm$ 5.8	92.2 $\pm$ 6.3	95.0 $\pm$ 7.4	87.7 $\pm$ 6.8	96.5 $\pm$ 0.4	93.2 $\pm$ 5.0	91.2 $\pm$ 7.3	87.6 $\pm$ 7.2
SDGCA [5]	82.2 $\pm$ 3.0	73.8 $\pm$ 4.1	50.9 $\pm$ 0.3	93.5 $\pm$ 3.9	15.1 $\pm$ 0.9	84.2 $\pm$ 2.2	78.6 $\pm$ 3.1	T/O	T/O	OOM
GPEC [8]	94.8 $\pm$ 3.8	89.6 $\pm$ 3.3	96.0 $\pm$ 2.7	96.2 $\pm$ 4.1	81.1 $\pm$ 6.9	94.0 $\pm$ 3.3	95.4 $\pm$ 0.9	T/O	T/O	OOM
FCE [10]	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0
TBAWEC [37]	100.0 $\pm$ 0.0	98.5 $\pm$ 1.3	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	T/O	T/O	T/O	T/O
OLEC-SB (Ours)	88.8 $\pm$ 4.3	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	75.1 $\pm$ 6.2	79.6 $\pm$ 1.0	96.9 $\pm$ 1.2	99.7 $\pm$ 0.0	46.1 $\pm$ 5.2	41.2 $\pm$ 3.4	59.5 $\pm$ 2.1

T/O indicates excessive runtime (mean execution time over 10 runs > 5000 s), and OOM indicates insufficient memory. Light-gray rows denote the balance-aware baselines FCE and TBAWEC, while the dark-gray row denotes the proposed OLEC-SB.

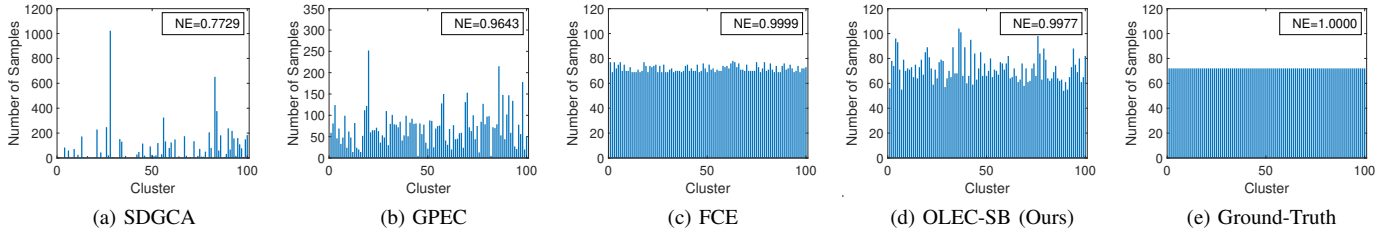


Fig. 4. Distribution of samples in each cluster on D7 in a single experimental run.

performance of OLEC-SB shows an increasing trend as the ensemble size increases. since more base clusterings provide richer complementary information, while the adaptive weighting strategy reduces the influence of less reliable ones. The improvement gradually becomes limited when additional base clusterings provide little complementary information. In addition, OLEC-SB consistently outperforms the compared methods across different ensemble sizes, except on D1, D3, and D6, demonstrating its robustness to ensemble-size selection. Notably, even when most methods achieve their best results with 25 base clusterings, they still underperform OLEC-SB with only 10 base clusterings on D2, D4, D5, and D7-D10.

F. Convergence Analysis

In Section III-D3, we prove that Algorithm 1 monotonically decreases the objective value of problem (6) and converges. This is also reflected in Figure 6, where OLEC-SB shows a clear decreasing trend and reaches stability within 10 outer iterations on D1-D10. On D7-D10, the average t_Y values are 8.31, 4.32, 3.28, and 4.79, respectively, much smaller than $n = 7200, 10845, 17898$, and 35040 , empirically supporting $t_Y \ll n$. This convergence behavior further explains Observation (iv) in Section IV-B: although OLEC-SB requires alternating updates of $\mathbf{Y}$ and α , only a few iterations are required, keeping the extra cost moderate in practice. More importantly, these updates enable adaptive weighting and

soft-balanced label learning, allowing OLEC-SB to improve clustering quality while maintaining practical efficiency.

G. Parameter Analysis

There are two hyper-parameters λ and γ in OLEC-SB, and we investigated their effects on OLEC-SB in Figure 7.

The effect of λ (cluster-size bias correction). As shown in Figure 7 (a)-(j), increasing λ improves the balance performance measured by NE, while its effect on ACC depends on the cluster-size distribution. On the balanced datasets D2, D3 and D7, a moderate increase in λ initially improves ACC, since balanced regularization helps alleviate degenerate partitions caused by large-cluster bias; however, when λ becomes too large, OLEC-SB may overcorrect the bias toward balanced cluster sizes and lead to a decrease in ACC. For the imbalanced datasets D1, D4-D6, and D8-D10, the best ACC is usually achieved with a smaller λ ; however, when λ is too close to zero, such as $\lambda = 10^{-7}$, ACC drops on all these datasets, indicating that cluster-size bias still exists in imbalanced data. Overall, OLEC-SB can handle both balanced and imbalanced datasets by tuning λ , and in most cases the best performance is obtained with $\lambda \in [10^{-6}, 10^{-5}, \dots, 10^{-1}]$.

The effect of γ (sharpness of adaptive weighting). As shown in Figure 7 (k)-(t), we vary γ from 1.1 to 3.0 with a step size of 0.1. The results show that γ should be properly selected. When γ is close to 1, the learned weights concentrate

TABLE VII
ABLATION STUDY OF OLEC-SB ON DIFFERENT DATASETS IN TERMS OF ACC, ARI, AND F-SCORE (MEAN $\pm$ STD, %), WHERE W/O DENOTES A VARIANT WITHOUT THE CORRESPONDING COMPONENT. BEST RESULTS ARE HIGHLIGHTED IN **BOLD**.

Method	Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	Avg
w/o normalization	ACC $\uparrow$	50.5 $\pm$ 5.6	22.4 $\pm$ 7.5	73.7 $\pm$ 5.8	47.1 $\pm$ 3.3	81.0 $\pm$ 10.4	59.6 $\pm$ 13.7	19.1 $\pm$ 8.9	48.1 $\pm$ 8.4	90.6 $\pm$ 8.1	49.0 $\pm$ 2.5	54.1 $\pm$ 7.4
	ARI $\uparrow$	8.9 $\pm$ 4.9	8.9 $\pm$ 4.1	23.7 $\pm$ 10.3	1.5 $\pm$ 2.1	33.1 $\pm$ 28.3	18.2 $\pm$ 19.1	14.3 $\pm$ 8.0	9.4 $\pm$ 8.5	47.9 $\pm$ 18.6	1.8 $\pm$ 1.3	16.8 $\pm$ 10.5
	F-score $\uparrow$	45.4 $\pm$ 3.1	25.4 $\pm$ 2.6	62.1 $\pm$ 5.1	48.6 $\pm$ 3.2	78.1 $\pm$ 11.4	57.7 $\pm$ 8.8	15.8 $\pm$ 7.8	48.7 $\pm$ 7.3	89.8 $\pm$ 8.5	50.1 $\pm$ 2.0	52.2 $\pm$ 6.0
w/o balancing	ACC $\uparrow$	41.7 $\pm$ 3.0	28.9 $\pm$ 7.6	53.2 $\pm$ 2.4	49.5 $\pm$ 0.9	80.9 $\pm$ 3.1	59.9 $\pm$ 1.4	32.5 $\pm$ 1.9	70.2 $\pm$ 2.4	91.4 $\pm$ 0.9	51.7 $\pm$ 0.2	56.0 $\pm$ 2.4
	ARI $\uparrow$	1.4 $\pm$ 2.3	9.0 $\pm$ 4.9	-0.4 $\pm$ 0.4	0.3 $\pm$ 0.6	5.3 $\pm$ 16.4	2.1 $\pm$ 2.6	29.4 $\pm$ 2.0	11.7 $\pm$ 13.0	9.5 $\pm$ 13.8	0.1 $\pm$ 0.1	6.8 $\pm$ 5.6
	F-score $\uparrow$	50.5 $\pm$ 0.5	25.3 $\pm$ 3.0	64.5 $\pm$ 3.1	54.4 $\pm$ 0.8	81.5 $\pm$ 2.1	62.5 $\pm$ 1.2	30.4 $\pm$ 1.9	68.2 $\pm$ 2.5	91.4 $\pm$ 0.7	55.6 $\pm$ 0.1	58.4 $\pm$ 1.6
w/o weighting	ACC $\uparrow$	46.4 $\pm$ 1.3	43.8 $\pm$ 3.4	86.0 $\pm$ 6.0	34.9 $\pm$ 0.4	64.0 $\pm$ 5.0	35.8 $\pm$ 0.7	31.0 $\pm$ 1.5	33.5 $\pm$ 3.7	55.9 $\pm$ 1.3	34.1 $\pm$ 0.1	46.5 $\pm$ 2.3
	ARI $\uparrow$	4.5 $\pm$ 0.6	22.1 $\pm$ 2.9	53.1 $\pm$ 16.7	0.1 $\pm$ 0.0	8.7 $\pm$ 3.3	0.2 $\pm$ 0.1	21.9 $\pm$ 1.5	9.7 $\pm$ 1.0	1.5 $\pm$ 0.6	0.0 $\pm$ 0.0	12.2 $\pm$ 2.7
	F-score $\uparrow$	36.7 $\pm$ 0.4	30.7 $\pm$ 2.6	76.7 $\pm$ 8.3	35.7 $\pm$ 0.0	61.3 $\pm$ 1.4	39.0 $\pm$ 0.1	22.7 $\pm$ 1.4	32.4 $\pm$ 0.8	63.1 $\pm$ 0.2	35.8 $\pm$ 0.0	43.4 $\pm$ 1.5
OLEC-SB (Ours)	ACC $\uparrow$	60.3 $\pm$ 3.0	61.5 $\pm$ 3.4	88.6 $\pm$ 4.7	56.1 $\pm$ 2.7	95.5 $\pm$ 0.6	69.0 $\pm$ 2.9	52.0 $\pm$ 1.5	72.2 $\pm$ 2.5	97.1 $\pm$ 0.8	64.1 $\pm$ 0.9	71.6 $\pm$ 2.3
	ARI $\uparrow$	22.8 $\pm$ 4.8	40.7 $\pm$ 3.4	60.3 $\pm$ 13.3	12.0 $\pm$ 3.4	80.9 $\pm$ 2.2	38.7 $\pm$ 2.0	44.0 $\pm$ 1.5	32.4 $\pm$ 7.7	79.2 $\pm$ 4.1	27.6 $\pm$ 2.4	43.9 $\pm$ 4.5
	F-score $\uparrow$	51.8 $\pm$ 3.2	47.3 $\pm$ 3.0	80.2 $\pm$ 6.6	51.0 $\pm$ 2.2	93.4 $\pm$ 0.8	63.4 $\pm$ 1.4	44.6 $\pm$ 1.5	70.4 $\pm$ 2.9	96.7 $\pm$ 0.9	60.5 $\pm$ 1.0	65.9 $\pm$ 2.4

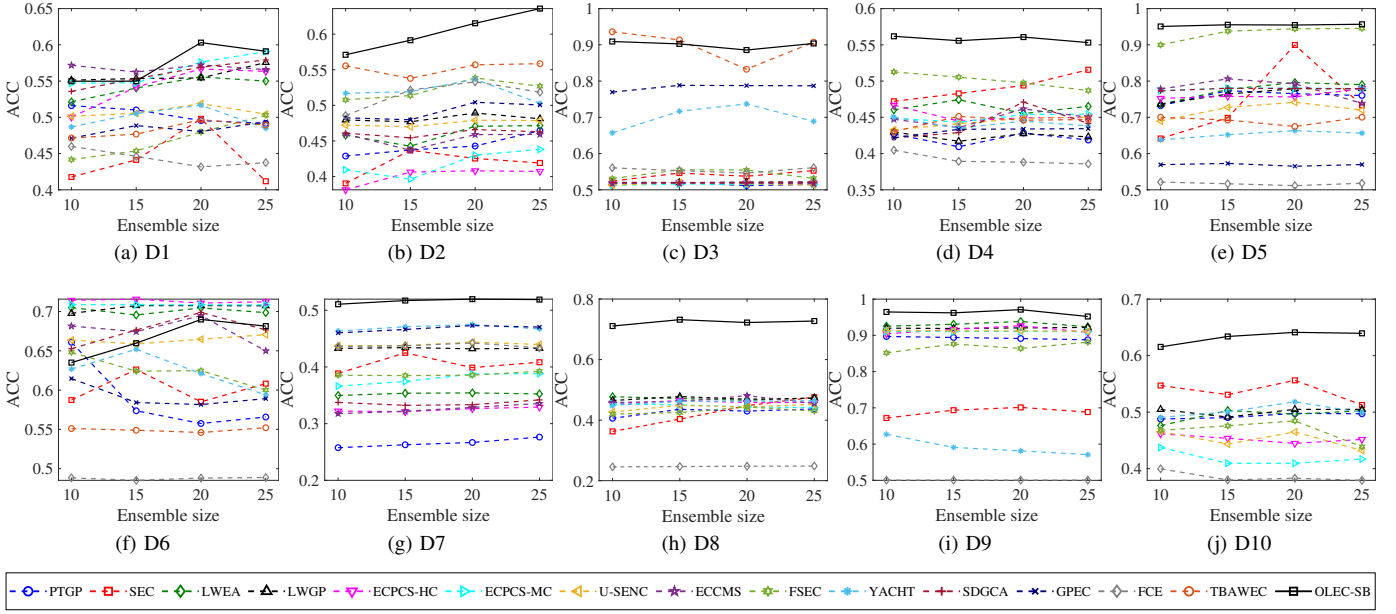


Fig. 5. Clustering performances of different methods by varying ensemble size *w.r.t.* ACC.

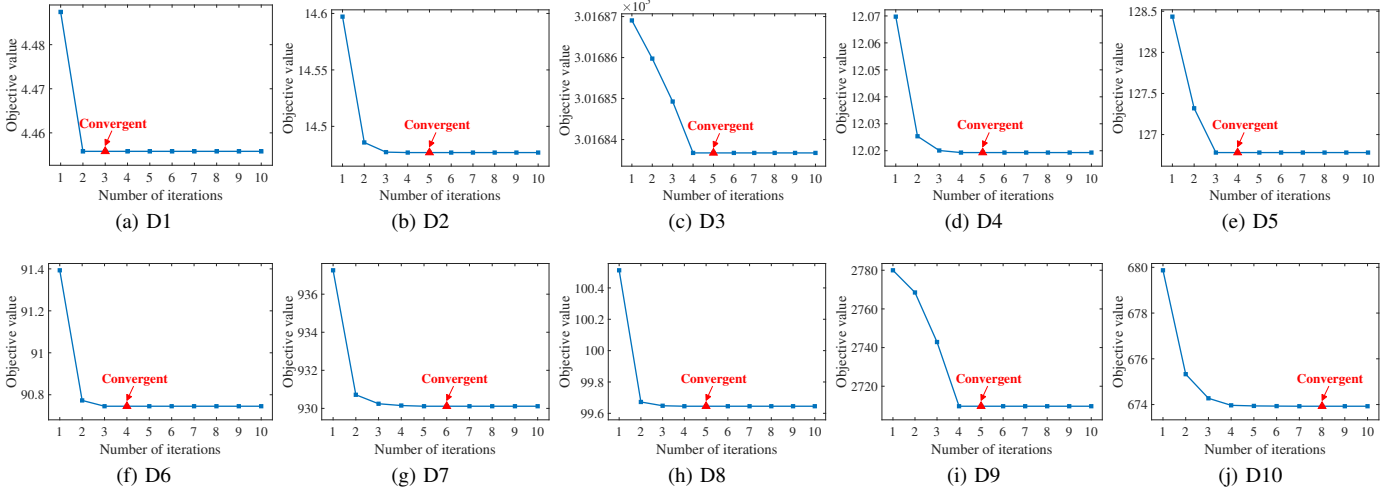


Fig. 6. Convergent curves of OLEC-SB on datasets D1-D10.

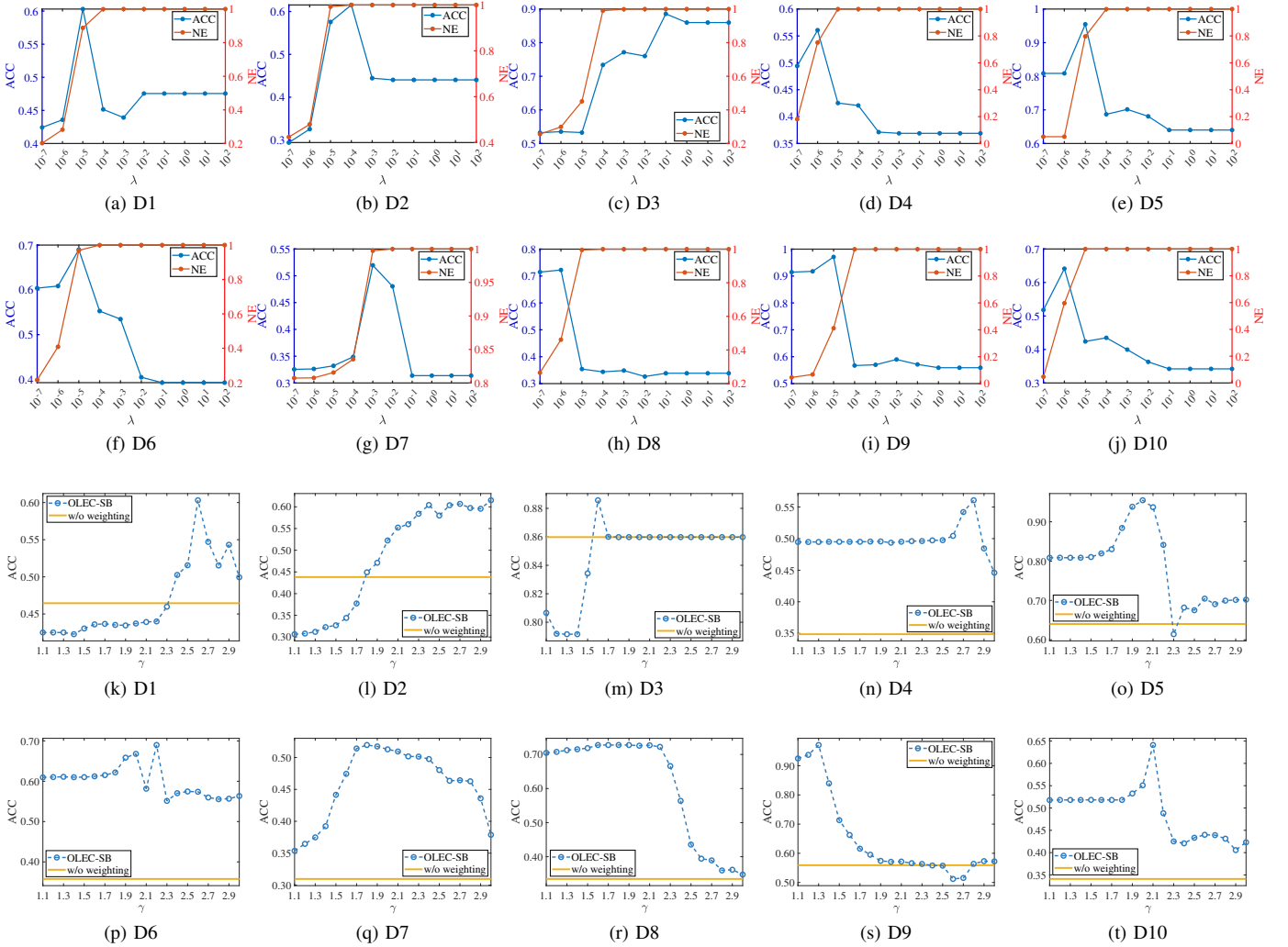


Fig. 7. Parameter sensitivity of OLEC-SB on D1-D10: (a)-(j) ACC and NE under different λ ; (k)-(t) ACC of OLEC-SB and w/o weighting under different γ .

on a few base clusterings, which may reduce ensemble robustness. When γ is too large, the weights become nearly uniform and the performance approaches the variant w/o weighting (fixing $\alpha_k = 1/m$). Therefore, we tune γ in $[1.1, 1.2, \dots, 3.0]$ and select the best value for each dataset.

To further validate OLEC-SB, we provide statistical significance analysis in the supplementary material.

V. CONCLUSION AND DISCUSSION

In this paper, we propose OLEC-SB, a mismatch-free one-step ensemble clustering framework that integrates discrete label learning and soft balancing under a linear-time optimization scheme. Specifically, OLEC-SB builds a normalized bipartite representation with cluster-size information. Based on this representation, OLEC-SB formulates consensus learning as a super-node similarity minimization problem with balanced regularization, and introduces an adaptive weighting strategy to capture the importance of each base clustering. To solve the discrete optimization problem efficiently, we develop a fast coordinate descent algorithm with three acceleration strategies,

achieving an overall complexity of $\mathcal{O}(nm + t(nKmt_Y + nm))$, which is linear in n when $K, m, t, t_Y \ll n$. We further provide theoretical guarantees for OLEC-SB by analyzing the doubly stochastic property of the normalized CA matrix, its monotonic convergence, averaged normalized-cut interpretation, and optimization-induced cluster-size bias correction. Extensive experiments on medium-scale and large-scale datasets show the superior effectiveness and efficiency of OLEC-SB over SOTA methods. Despite these advantages, OLEC-SB requires dataset-specific tuning of two hyper-parameters and does not explicitly consider group-level fairness with respect to protected attributes. In future work, we will develop data-driven self-tuning strategies and extend OLEC-SB to fairness-aware ensemble clustering while preserving its scalability.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grants U24A20322 and 62576094. This research work was also supported by the Big Data Computing Center of Southeast University.

Generative AI tools, including ChatGPT, were used only for language polishing, grammar checking, and formatting assistance during manuscript preparation.

REFERENCES

- [1] A. L. Fred and A. K. Jain, "Combining multiple clusterings using evidence accumulation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 6, pp. 835–850, 2005.
- [2] C. Wang, Z. She, and L. Cao, "Coupled clustering ensemble: Incorporating coupling relationships both between base clusterings and objects," in *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 2013, pp. 374–385.
- [3] D. Treder-Tschechlov, M. Fritz, H. Schwarz, and B. Mitschang, "Ensemble clustering based on meta-learning and hyperparameter optimization," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 2880–2892, 2024.
- [4] A. Strehl and J. Ghosh, "Cluster ensembles—a knowledge reuse framework for combining multiple partitions," *Journal of Machine Learning Research*, vol. 3, no. Dec, pp. 583–617, 2002.
- [5] X. Zhang, Y. Jia, M. Song, and R. Wang, "Similarity and dissimilarity guided co-association matrix construction for ensemble clustering," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 11, pp. 6694–6707, 2025.
- [6] D. Huang, C.-D. Wang, and J.-H. Lai, "Locally weighted ensemble clustering," *IEEE Transactions on Cybernetics*, vol. 48, no. 5, pp. 1460–1473, 2017.
- [7] Y. Jia, S. Tao, R. Wang, and Y. Wang, "Ensemble clustering via co-association matrix self-enhancement," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 8, pp. 11 168–11 179, 2023.
- [8] X. Zhang, H. Qiu, W. Liang, H. Liu, J. Hou, and Y. Jia, "Generalization performance of ensemble clustering: from theory to algorithm," in *Proceedings of the Forty-Second International Conference on Machine Learning*, vol. 267, 2025.
- [9] T. Qi, H. Qiu, Y. Jia, X. Shi, and R. Wang, "Reliable co-occurrence guided partial clustering ensemble," *IEEE Transactions on Neural Networks and Learning Systems*, 2026.
- [10] P. Zhou, R. Li, Z. Ling, L. Du, and X. Liu, "Fair clustering ensemble with equal cluster capacity," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 3, pp. 1729–1746, 2025.
- [11] M. S. Mahmud, J. Z. Huang, R. Ruby, A. Nguetilbaye, and K. Wu, "Approximate clustering ensemble method for big data," *IEEE Transactions on Big Data*, vol. 9, no. 4, pp. 1142–1155, 2023.
- [12] D. Huang, J.-H. Lai, and C.-D. Wang, "Robust ensemble clustering using probability trajectories," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 5, pp. 1312–1326, 2015.
- [13] D. Huang, C.-D. Wang, J.-S. Wu, J.-H. Lai, and C.-K. Kwoh, "Ultra-scalable spectral clustering and ensemble clustering," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 6, pp. 1212–1226, 2019.
- [14] H. Li, X. Ye, A. Imakura, and T. Sakurai, "Lsec: large-scale spectral ensemble clustering," *Intelligent Data Analysis*, vol. 27, no. 1, pp. 59–77, 2023.
- [15] M. S. Mahmud, J. Z. Huang, and S. García, "Clustering approximation via a fusion of multiple random samples," *Information Fusion*, vol. 101, p. 101986, 2024.
- [16] M. S. Mahmud, H. Zheng, D. Garcia-Gil, S. Garcia, and J. Z. Huang, "Rspca: random sample partition and clustering approximation for ensemble learning of big data," *Pattern Recognition*, vol. 161, no. C, p. 111321, 2025.
- [17] D. Huang, C.-D. Wang, H. Peng, J. Lai, and C.-K. Kwoh, "Enhanced ensemble clustering via fast propagation of cluster-wise similarities," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 508–520, 2018.
- [18] H. Yu, Y. Chen, P. Lingras, and G. Wang, "A three-way cluster ensemble approach for large-scale data," *International Journal of Approximate Reasoning*, vol. 115, no. C, pp. 32–49, 2019.
- [19] Y. Liang, Z. Ren, Z. Wu, D. Zeng, and J. Li, "Scalable spectral ensemble clustering via building representative co-association matrix," *Neurocomputing*, vol. 390, pp. 158–167, 2020.
- [20] G. Yang, S. Deng, C. Chen, Y. Yang, Z. Gong, X. Chen, and Z. Hao, "Litewsec: a lightweight framework for web-scale spectral ensemble clustering," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 10, pp. 10035–10047, 2023.
- [21] J. Xu, L. Duan, X. Wang, and L. Du, "Dynamic anchor-based ensemble clustering via hypergraph reconstruction," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025, pp. 6740–6748.
- [22] R. Zhang, S. Hang, Z. Sun, F. Nie, R. Wang, and X. Li, "Anchor-based fast spectral ensemble clustering," *Information Fusion*, vol. 113, no. C, p. 102587, 2025.
- [23] G. Zhong and C.-M. Pun, "A unified framework for multi-view spectral clustering," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020, pp. 1854–1857.
- [24] P. Zhou, L. Du, and X. Li, "Self-paced consensus clustering with bipartite graph," in *Proceedings of the Twenty-Ninth International Conference on Artificial Intelligence*, 2021, pp. 2133–2139.
- [25] P. Zhou, X. Liu, L. Du, and X. Li, "Self-paced adaptive bipartite graph learning for consensus clustering," *ACM Transactions on Knowledge Discovery from Data*, vol. 17, no. 5, pp. 1–35, 2023.
- [26] Z. Zhang, X. Chen, C. Wang, R. Wang, W. Song, and F. Nie, "A structured bipartite graph learning method for ensemble clustering," *Pattern Recognition*, vol. 160, p. 111133, 2025.
- [27] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [28] P. Zhou, L. Du, X. Liu, Z. Ling, X. Ji, X. Li, and Y.-D. Shen, "Partial clustering ensemble," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 5, pp. 2096–2109, 2023.
- [29] Y. Jia, S. Kwong, and J. Hou, "Semi-supervised spectral clustering with structured sparsity regularization," *IEEE Signal Processing Letters*, vol. 25, no. 3, pp. 403–407, 2018.
- [30] Y. Jia, H. Liu, J. Hou, S. Kwong, and Q. Zhang, "Multi-view spectral clustering tailored tensor low-rank representation," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 12, pp. 4784–4797, 2021.
- [31] H. Liu, T. Liu, J. Wu, D. Tao, and Y. Fu, "Spectral ensemble clustering," in *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2015, pp. 715–724.
- [32] Z. Bian, J. Qu, Z. Deng, and S. Wang, "One-step fuzzy ensemble clustering method via embedding ground-truth cluster number graphs," *IEEE Transactions on Fuzzy Systems*, vol. 33, no. 9, pp. 3225–3239, 2025.
- [33] P. S. Bradley, K. P. Bennett, and A. Demiriz, "Constrained k-means clustering," *Microsoft Research, Redmond*, vol. 20, no. 0, p. 0, 2000.
- [34] M. I. Malinen and P. Fränti, "Balanced k-means for clustering," in *Joint IAPR International Workshops on Statistical Techniques in Pattern Recognition (SPR) and Structural and Syntactic Pattern Recognition (SSPR)*. Springer, 2014, pp. 32–41.
- [35] Z. Li, F. Nie, X. Chang, Z. Ma, and Y. Yang, "Balanced clustering via exclusive lasso: a pragmatic approach," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [36] Y. Ji, S. Zhu, S. Huang, Z. Liu, S. Wang, and Z. Peng, "Federated and balanced clustering for high-dimensional data," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4032–4044, 2025.
- [37] R. Zhang, X. Wu, H. Chen, G. He, Z. Wang, R. Wang, and F. Nie, "Toward balance adaptive weighted ensemble clustering," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 35, no. 6, pp. 5279–5292, 2025.
- [38] C. K. Reddy and B. Vinzamuri, "A survey of partitional and hierarchical clustering algorithms," in *Data Clustering*. Chapman and Hall/CRC, 2018, pp. 87–110.
- [39] Y. Cai, M. S. Mahmud, J. Xu, X. Sun, and J. Z. Huang, "Spectral ensemble clustering with doubly stochastic co-association matrix," *Information Sciences*, vol. 686, no. C, p. 121314, 2025.
- [40] P. Tseng, "Convergence of a block coordinate descent method for nondifferentiable minimization," *Journal of Optimization Theory and Applications*, vol. 109, no. 3, pp. 475–494, 2001.
- [41] S. J. Wright, "Coordinate descent algorithms," *Mathematical Programming*, vol. 151, no. 1, pp. 3–34, 2015.
- [42] H. Qiu, Z. Liu, H. Huang, S. Letchmunan, M. Deveci, and T. Senapati, "L2-regularization based two-way weighted neutrosophic clustering with manhattan and euclidean distances," *Fuzzy Sets and Systems*, vol. 518, p. 109507, 2025.

OVERVIEW OF THE SUPPLEMENTAL MATERIAL

Our supplemental material consists of three main sections:

- **Detailed Proofs:** proofs of Theorem 1, Theorem 2, Theorem 3, Theorem 4, Section III-E1, and Theorem 5.
- **Complexity Analysis:** space complexity analysis of OLEC-SB.
- **Additional Experiments:** supplementary experimental results, including dataset details, compared methods, clustering performance (ARI/F-score), and statistical analysis.

SUPPLEMENTAL MATERIAL A DETAILED PROOF

In this section, we provide detailed proofs for the theoretical results presented in the main paper, including Theorem 1, Theorem 2, Theorem 3, Theorem 4, Section III-E1, and Theorem 5.

A. Proof of Theorem 1

Proof. By the Cauchy–Schwarz inequality,

$$\left(\sum_{l=1}^K |\mathcal{C}_l|^2 \right) \left(\sum_{l=1}^K 1^2 \right) \geq \left(\sum_{l=1}^K |\mathcal{C}_l| \cdot 1 \right)^2. \quad (\text{S1})$$

Since $\sum_{l=1}^K 1^2 = K$ and $\sum_{l=1}^K |\mathcal{C}_l| = n$, we obtain

$$\sum_{l=1}^K |\mathcal{C}_l|^2 \geq \frac{n^2}{K}, \quad (\text{S2})$$

and the equality holds if and only if $|\mathcal{C}_1| = \dots = |\mathcal{C}_K| = n/K$. This completes the proof of Theorem 1. $\square$

B. Proof of Theorem 2

Proof. By Eq. (1), since $\bar{\mathcal{C}}_l = \bigcup_{h \neq l} \mathcal{C}_h$ and the clusters are pairwise disjoint, the similarity between $\mathcal{C}_l$ and its complement $\bar{\mathcal{C}}_l$ can be decomposed as the sum of the similarities between $\mathcal{C}_l$ and each of the remaining clusters. Thus, we have

$$\text{sim}_k(\mathcal{C}_l, \bar{\mathcal{C}}_l) = \sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) = \sum_{i \in \mathcal{C}_l} \sum_{j \notin \mathcal{C}_l} w_{ij}^k. \quad (\text{S3})$$

On the other hand, let $\mathbf{y}_l$ denote the indicator vector of cluster $\mathcal{C}_l$, i.e., $(\mathbf{y}_l)_i = 1$ if $x_i \in \mathcal{C}_l$, and $(\mathbf{y}_l)_i = 0$ otherwise. Since $\mathbf{D}_k$ is the diagonal degree matrix associated with $\mathbf{W}_k$, the quadratic form $\mathbf{y}_l^\top \mathbf{D}_k \mathbf{y}_l$ selects precisely the degree terms corresponding to the samples in $\mathcal{C}_l$. Thus,

$$\mathbf{y}_l^\top \mathbf{D}_k \mathbf{y}_l = \sum_{i=1}^n (\mathbf{y}_l)_i^2 d_i^k = \sum_{i \in \mathcal{C}_l} d_i^k = \sum_{i \in \mathcal{C}_l} \sum_{j=1}^n w_{ij}^k. \quad (\text{S4})$$

Similarly,

$$\mathbf{y}_l^\top \mathbf{W}_k \mathbf{y}_l = \sum_{i=1}^n \sum_{j=1}^n (\mathbf{y}_l)_i w_{ij}^k (\mathbf{y}_l)_j = \sum_{i \in \mathcal{C}_l} \sum_{j \in \mathcal{C}_l} w_{ij}^k. \quad (\text{S5})$$

Therefore,

$$\mathbf{y}_l^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{y}_l = \sum_{i \in \mathcal{C}_l} \sum_{j=1}^n w_{ij}^k - \sum_{i \in \mathcal{C}_l} \sum_{j \in \mathcal{C}_l} w_{ij}^k = \sum_{i \in \mathcal{C}_l} \sum_{j \notin \mathcal{C}_l} w_{ij}^k. \quad (\text{S6})$$

Combining the above equality with the definition of inter-cluster similarity yields

$$\sum_{h \neq l} \text{sim}_k(\mathcal{C}_l, \mathcal{C}_h) = \mathbf{y}_l^\top (\mathbf{D}_k - \mathbf{W}_k) \mathbf{y}_l. \quad (\text{S7})$$

This completes the proof of Theorem 2. $\square$

C. Proof of Theorem 3

Proof. Since $\Delta_k = \mathbf{A}_k^\top \mathbf{A}_k$ is diagonal with strictly positive diagonal entries, Δ_k^{-1} exists. For any sample $\mathbf{x}_i$, denote by $r(i)$ the cluster index such that $(\mathbf{A}_k)_{i r(i)} = 1$. From Eq. (5), the (i, j) -th entry of $\mathbf{W}_k$ can be written as

$$\begin{aligned} (\mathbf{W}_k)_{ij} &= \sum_{r=1}^{l_k} (\mathbf{A}_k)_{ir} (\Delta_k^{-1})_{rr} (\mathbf{A}_k)_{jr} \\ &= \begin{cases} 1/|\mathcal{C}_{r(i)}^k|, & \text{if } \mathbf{x}_j \in \mathcal{C}_{r(i)}^k, \\ 0, & \text{otherwise,} \end{cases} \end{aligned} \quad (\text{S8})$$

where $|\mathcal{C}_{r(i)}^k|$ denotes the size of the cluster containing $\mathbf{x}_i$ under π_k . Therefore, the i -th row sum is

$$\sum_{j=1}^n (\mathbf{W}_k)_{ij} = \sum_{j \in \mathcal{C}_{r(i)}^k} \frac{1}{|\mathcal{C}_{r(i)}^k|} = 1, \quad (\text{S9})$$

which implies $\mathbf{W}_k \mathbf{1}_n = \mathbf{1}_n$. Moreover, $\mathbf{W}_k$ is symmetric because Δ_k^{-1} is diagonal and $\mathbf{W}_k = \mathbf{A}_k \Delta_k^{-1} \mathbf{A}_k^\top$, hence $\mathbf{1}_n^\top \mathbf{W}_k = (\mathbf{W}_k \mathbf{1}_n)^\top = \mathbf{1}_n^\top$. Thus, $\mathbf{W}_k$ is doubly stochastic. Finally, by definition $(\mathbf{D}_k)_{ii} = \sum_{j=1}^n w_{ij}^k$, we obtain $(\mathbf{D}_k)_{ii} = 1$ for all i , i.e., $\mathbf{D}_k = \mathbf{I}$. This completes the proof of Theorem 3. $\square$

D. Proof of Theorem 4

Proof. Since $\mathbf{W}_k = \mathbf{B}_k \mathbf{B}_k^\top$ is doubly stochastic (by Theorem 3), it holds that $\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top \succeq \mathbf{0}$, and thus $\text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}) \geq 0$ for all k . For $\gamma > 1$, problem (21) is convex in α over the probability simplex, so the KKT conditions are sufficient for global optimality.

By introducing the Lagrange multipliers β for $\mathbf{1}_m^\top \alpha = 1$ and $\mu_k \geq 0$ for $\alpha_k \geq 0$, the Lagrangian function is given by

$$\begin{aligned} \mathcal{L}(\alpha, \beta, \mu) &= \sum_{k=1}^m \alpha_k^\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}) - \beta (\mathbf{1}_m^\top \alpha - 1) - \mu^\top \alpha. \end{aligned} \quad (\text{S10})$$

Taking the derivative of $\mathcal{L}$ w.r.t. α_k and setting it to zero yields

$$\frac{\partial \mathcal{L}}{\partial \alpha_k} = \gamma \alpha_k^{\gamma-1} \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}) - \beta - \mu_k = 0. \quad (\text{S11})$$

Because $\sum_{k=1}^m \alpha_k = 1$, there exists j such that $\alpha_j > 0$, which implies $\mu_j = 0$ and thus

$$\beta = \gamma \alpha_j^{\gamma-1} \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_j \mathbf{B}_j^\top) \mathbf{Y}) > 0. \quad (\text{S12})$$

This further indicates that $\alpha_k = 0$ cannot be optimal: if $\alpha_k = 0$ for some k , then Eq. (S11) reduces to $-\beta - \mu_k = 0$, which yields $\mu_k = -\beta < 0$ and violates $\mu_k \geq 0$. Therefore, the

optimum satisfies $\alpha_k > 0$ for all k , and hence $\mu_k = 0$ for all k . Substituting $\mu_k = 0$ into Eq. (S11) yields

$$\gamma \alpha_k^{\gamma-1} \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y}) = \beta, \quad (\text{S13})$$

which leads to

$$\alpha_k = \left(\frac{\beta}{\gamma \text{tr}(\mathbf{Y}^\top (\mathbf{I} - \mathbf{B}_k \mathbf{B}_k^\top) \mathbf{Y})} \right)^{\frac{1}{\gamma-1}}. \quad (\text{S14})$$

Finally, enforcing $\sum_{k=1}^m \alpha_k = 1$ eliminates β and yields the closed-form update in Eq. (22). This completes the proof of Theorem 4. $\square$

E. Proof of Theorem III-E1

Proof. Let $\mathcal{L}(\mathbf{Y}, \boldsymbol{\alpha})$ denote the objective in problem (6). In t -th iteration, given $(\mathbf{Y}^{(t)}, \boldsymbol{\alpha}^{(t)})$, Algorithm 1 updates $\mathbf{Y}$ and $\boldsymbol{\alpha}$ by alternately solving the two corresponding subproblems.

Update $\mathbf{Y}$. With $\boldsymbol{\alpha}^{(t)}$ fixed, the $\mathbf{Y}$ -subproblem is solved to obtain $\mathbf{Y}^{(t+1)}$. By optimality,

$$\mathcal{L}(\mathbf{Y}^{(t+1)}, \boldsymbol{\alpha}^{(t)}) \leq \mathcal{L}(\mathbf{Y}^{(t)}, \boldsymbol{\alpha}^{(t)}). \quad (\text{S15})$$

Update $\boldsymbol{\alpha}$. With $\mathbf{Y}^{(t+1)}$ fixed, the $\boldsymbol{\alpha}$ -subproblem is solved to obtain $\boldsymbol{\alpha}^{(t+1)}$. Similarly,

$$\mathcal{L}(\mathbf{Y}^{(t+1)}, \boldsymbol{\alpha}^{(t+1)}) \leq \mathcal{L}(\mathbf{Y}^{(t+1)}, \boldsymbol{\alpha}^{(t)}). \quad (\text{S16})$$

Combining (S15) and (S16) yields the monotonic descent property

$$\mathcal{L}(\mathbf{Y}^{(t+1)}, \boldsymbol{\alpha}^{(t+1)}) \leq \mathcal{L}(\mathbf{Y}^{(t)}, \boldsymbol{\alpha}^{(t)}). \quad (\text{S17})$$

Moreover, $\boldsymbol{\alpha}$ is restricted to the probability simplex $\{\boldsymbol{\alpha} \succeq \mathbf{0} \mid \mathbf{1}_m^\top \boldsymbol{\alpha} = 1\}$ and $\mathbf{Y} \in \text{Ind}$ by construction; hence the sequence $\{\mathcal{L}(\mathbf{Y}^{(t)}, \boldsymbol{\alpha}^{(t)})\}$ is bounded below. Thus, Algorithm 1 converges. This completes the proof of Theorem III-E1. $\square$

F. Proof of Theorem 5

Proof. By Eq. (12), the row-wise update of $\mathbf{x}_i$ is determined by minimizing $\varphi(s)$ over $s \in \{1, \dots, K\}$, where $\varphi(s)$ denotes the objective difference between assigning $\mathbf{x}_i$ to the s -th cluster and the auxiliary zero-row state.

Since $\mathbf{Y}^{(s)}$ and $\mathbf{Y}^{(0)}$ differ only in the assignment of $\mathbf{x}_i$, only the s -th column changes. Therefore, the balancing term satisfies

$$\begin{aligned} (\mathbf{y}_s^{(s)})^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{y}_s^{(s)} - (\mathbf{y}_s^{(0)})^\top \mathbf{1}_n \mathbf{1}_n^\top \mathbf{y}_s^{(0)} \\ = 2(\mathbf{y}_s^{(0)})^\top \mathbf{1}_n + 1 = 2|\mathcal{C}_s^{-i}| + 1. \end{aligned} \quad (\text{S18})$$

Similarly, for each k , expanding the quadratic form gives

$$\begin{aligned} (\mathbf{y}_s^{(s)})^\top \mathbf{B}_k \mathbf{B}_k^\top \mathbf{y}_s^{(s)} - (\mathbf{y}_s^{(0)})^\top \mathbf{B}_k \mathbf{B}_k^\top \mathbf{y}_s^{(0)} \\ = 2(\mathbf{y}_s^{(0)})^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top + (\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top. \end{aligned} \quad (\text{S19})$$

Substituting Eqs (S18)-(S19) into $\varphi(s)$, we obtain

$$\begin{aligned} \varphi(s) = -2 \left(\sum_{k=1}^m \alpha_k^\gamma (\mathbf{y}_s^{(0)})^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top - \lambda |\mathcal{C}_s^{-i}| \right) \\ + \lambda - \sum_{k=1}^m \alpha_k^\gamma (\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top. \end{aligned} \quad (\text{S20})$$

The last two terms are independent of s . Hence, minimizing $\varphi(s)$ is equivalent to maximizing

$$\sum_{k=1}^m \alpha_k^\gamma (\mathbf{y}_s^{(0)})^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top - \lambda |\mathcal{C}_s^{-i}|. \quad (\text{S21})$$

Next, using $\mathbf{W}_k = \mathbf{B}_k \mathbf{B}_k^\top$, we have

$$\sum_{k=1}^m \alpha_k^\gamma (\mathbf{y}_s^{(0)})^\top \mathbf{B}_k (\mathbf{b}_k)_i^\top = \sum_{k=1}^m \alpha_k^\gamma \sum_{j \in \mathcal{C}_s^{-i}} (\mathbf{W}_k)_{ji} = |\mathcal{C}_s^{-i}| \bar{a}_{is}, \quad (\text{S22})$$

where

$$\bar{a}_{is} = \frac{1}{|\mathcal{C}_s^{-i}|} \sum_{k=1}^m \alpha_k^\gamma \sum_{j \in \mathcal{C}_s^{-i}} (\mathbf{W}_k)_{ji}. \quad (\text{S23})$$

Hence, the row-wise update of $\mathbf{x}_i$ is

$$q = \arg \max_{s \in \{1, \dots, K\}} (|\mathcal{C}_s^{-i}| \bar{a}_{is} - \lambda |\mathcal{C}_s^{-i}|). \quad (\text{S24})$$

This completes the proof of Theorem 5. $\square$

SUPPLEMENTAL MATERIAL B COMPLEXITY ANALYSIS

In this section, we further discuss the space complexity of OLEC-SB. The indicator matrix $\mathbf{Y} \in \{0, 1\}^{n \times K}$ requires $\mathcal{O}(nK)$ space, and the weight vector $\boldsymbol{\alpha} \in \mathbb{R}^m$ costs $\mathcal{O}(m)$ space. For each base clustering, $\mathbf{B}_k \in \mathbb{R}^{n \times l_k}$ contains exactly one nonzero entry per row and can be stored in a sparse form, requiring $\mathcal{O}(n)$ memory per k and $\mathcal{O}(nm)$ in total. Moreover, to accelerate the update of $\mathbf{Y}$, we store $\mathbf{y}_s^\top \mathbf{B}_k$ and $\mathbf{y}_s^\top \mathbf{1}_n$ for all s and k , which needs $\mathcal{O}(K \sum_{k=1}^m l_k)$ and $\mathcal{O}(K)$ space, respectively. Finally, the term $(\mathbf{b}_k)_i (\mathbf{b}_k)_i^\top$ can be computed in $\mathcal{O}(1)$ time due to the one-sparse structure of $(\mathbf{b}_k)_i$, and thus does not need to be explicitly stored. Therefore, the overall space complexity of OLEC-SB is $\mathcal{O}(nK + nm + K \sum_{k=1}^m l_k)$, which is linear in the number of samples n .

SUPPLEMENTAL MATERIAL C ADDITIONAL EXPERIMENTS

In this section, we provide additional details about the comparative experiments that are omitted in the main paper due to space limitations.

A. Details of Datasets

In the experiments in Section IV, we used 10 benchmark datasets including images, sensor data, and textual information etc. We have summarized the feature information of the datasets in Table S1, and the detailed information is as follows:

- **Maternal Health Risk¹**: The dataset consists of clinical health data from pregnant patients, including parameters like blood pressure and blood sugar to predict risk levels.
- **CNAE-9²**: The dataset contains 1,080 documents categorized into nine distinct classes, representing various economic activities of Brazilian companies.

¹<https://archive.ics.uci.edu/dataset/863/maternal+health+risk>

²<https://archive.ics.uci.edu/dataset/233/cnae+9>

TABLE S1
DETAILS OF DIFFERENT DATASETS.

No.	Dataset	#Instance	#Feature	#Class	Scale	Cluster-Size Distributions
D1	Maternal Health Risk	1014	6	3	Medium	Imbalanced
D2	CNAE-9	1080	857	9	Medium	Balanced
D3	Phishing Websites	2456	30	2	Medium	Balanced
D4	PSDAS	4424	36	3	Medium	Imbalanced
D5	TUANDROMD	4464	241	2	Medium	Imbalanced
D6	Gender Gap	4746	18	3	Medium	Imbalanced
D7	COIL100	7200	1024	100	Large	Balanced
D8	Crowdsourced Mapping	10845	29	6	Large	Imbalanced
D9	HTRU2	17898	8	2	Large	Imbalanced
D10	Steel Energy	35040	9	3	Large	Imbalanced

TABLE S2
PERFORMANCE COMPARISON ON DIFFERENT DATASETS IN TERMS OF RUNNING TIME (SEC.) AND ARI (MEAN $\pm$ STD, %). BEST RESULTS ARE HIGHLIGHTED IN **BOLD**, AND SECOND-BEST RESULTS ARE MARKED WITH AN UNDERLINE.

Method	Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	Avg.
PTGP [12]	time↓	0.04	0.20	0.65	3.06	0.36	1.95	3.72	178.95	1601.57	51.33	184.18
	ARI↑	11.1 $\pm$ 5.2	19.1 $\pm$ 1.7	-0.3 $\pm$ 0.2	2.8 $\pm$ 3.2	6.4 $\pm$ 18.4	9.1 $\pm$ 9.4	24.2 $\pm$ 1.4	15.8 $\pm$ 2.2	46.4 $\pm$ 1.3	17.7 $\pm$ 5.4	15.2 $\pm$ 4.8
SEC [31]	time↓	0.00	0.01	0.01	0.03	0.01	0.03	0.79	0.16	0.13	0.47	0.16
	ARI↑	12.5 $\pm$ 2.4	19.2 $\pm$ 3.7	1.6 $\pm$ 6.0	2.2 $\pm$ 2.4	4.8 $\pm$ 30.0	9.1 $\pm$ 8.3	38.3 $\pm$ 1.2	6.7 $\pm$ 16.0	37.2 $\pm$ 29.1	2.3 $\pm$ 7.2	13.4 $\pm$ 10.6
LWEA [6]	time↓	0.25	0.30	1.14	3.31	3.61	3.28	5.51	15.16	53.73	152.54	23.88
	ARI↑	15.8 $\pm$ 0.8	21.4 $\pm$ 3.6	-0.6 $\pm$ 0.0	4.8 $\pm$ 2.7	6.5 $\pm$ 19.1	<u>31.2</u> $\pm$ 2.1	32.8 $\pm$ 1.2	<u>24.5</u> $\pm$ 6.8	63.2 $\pm$ 13.1	17.7 $\pm$ 5.4	21.7 $\pm$ 5.5
LWGP [6]	time↓	0.09	0.16	0.19	0.33	0.28	0.36	1.57	0.88	1.38	3.17	0.84
	ARI↑	19.3 $\pm$ 5.7	24.6 $\pm$ 2.2	-0.5 $\pm$ 0.0	1.7 $\pm$ 1.5	-2.7 $\pm$ 2.0	30.6 $\pm$ 0.6	39.5 $\pm$ 0.7	21.1 $\pm$ 6.1	55.7 $\pm$ 0.7	<u>20.0</u> $\pm$ 0.3	20.9 $\pm$ 2.0
ECPCS-HC [17]	time↓	0.22	0.70	1.19	3.62	3.72	14.21	17.69	45.00	67.77	262.42	41.66
	ARI↑	17.5 $\pm$ 1.8	11.0 $\pm$ 1.3	-0.5 $\pm$ 0.2	3.0 $\pm$ 2.1	-3.7 $\pm$ 3.3	30.4 $\pm$ 0.4	28.5 $\pm$ 1.8	15.2 $\pm$ 1.8	<u>58.4</u> $\pm$ 9.6	5.8 $\pm$ 9.9	16.5 $\pm$ 3.2
ECPCS-MC [17]	time↓	0.09	0.17	0.32	0.73	0.73	1.21	3.87	5.08	4.59	12.55	2.93
	ARI↑	<u>20.4</u> $\pm$ 6.1	11.9 $\pm$ 1.5	-0.3 $\pm$ 0.2	3.9 $\pm$ 3.2	-2.0 $\pm$ 0.4	30.8 $\pm$ 0.1	35.2 $\pm$ 1.2	13.5 $\pm$ 2.3	52.3 $\pm$ 1.6	3.4 $\pm$ 11.2	16.9 $\pm$ 2.8
U-SENC [13]	time↓	<u>0.02</u>	<u>0.04</u>	<u>0.06</u>	<u>0.13</u>	<u>0.10</u>	<u>0.13</u>	<u>0.85</u>	<u>0.47</u>	<u>0.85</u>	<u>1.71</u>	<u>0.43</u>
	ARI↑	13.5 $\pm$ 8.6	21.5 $\pm$ 1.2	-0.4 $\pm$ 0.2	4.1 $\pm$ 2.7	1.8 $\pm$ 9.2	30.9 $\pm$ 2.2	40.5 $\pm$ 1.0	15.3 $\pm$ 3.2	52.8 $\pm$ 1.2	13.0 $\pm$ 11.5	19.3 $\pm$ 4.1
ECCMS [7]	time↓	9.35	11.99	67.75	340.66	424.44	401.62	1422.21	4245.39	T/O	T/O	–
	ARI↑	18.9 $\pm$ 2.8	20.7 $\pm$ 3.6	-0.6 $\pm$ 0.1	4.7 $\pm$ 3.1	4.0 $\pm$ 19.0	28.2 $\pm$ 3.8	26.1 $\pm$ 3.3	24.0 $\pm$ 3.6	T/O	T/O	–
FSEC [22]	time↓	0.27	0.50	0.72	13.94	13.93	1.49	3.05	5.43	22.54	21.20	8.31
	ARI↑	9.9 $\pm$ 6.8	31.2 $\pm$ 2.9	1.3 $\pm$ 2.0	3.6 $\pm$ 1.0	<u>75.5</u> $\pm$ 2.9	25.8 $\pm$ 9.6	36.8 $\pm$ 0.8	17.3 $\pm$ 7.5	39.3 $\pm$ 4.8	9.3 $\pm$ 9.6	<u>25.0</u> $\pm$ 4.8
YACHT [21]	time↓	1.56	1.14	3.72	20.18	12.83	10.18	23.18	94.48	81.06	178.68	36.70
	ARI↑	12.6 $\pm$ 4.8	30.1 $\pm$ 3.2	28.3 $\pm$ 14.2	5.3 $\pm$ 3.8	18.6 $\pm$ 14.5	23.1 $\pm$ 11.1	44.3 $\pm$ 1.5	18.6 $\pm$ 8.4	-4.4 $\pm$ 2.3	12.8 $\pm$ 8.6	18.9 $\pm$ 7.3
SDGCA [5]	time↓	27.87	25.13	121.11	601.67	1283.63	1250.41	4525.16	T/O	T/O	OOM	–
	ARI↑	19.1 $\pm$ 5.9	21.2 $\pm$ 2.3	-0.6 $\pm$ 0.0	5.2 $\pm$ 2.7	-2.0 $\pm$ 0.2	29.2 $\pm$ 1.7	25.8 $\pm$ 4.3	T/O	T/O	OOM	–
GPEC [8]	time↓	62.97	100.08	231.57	777.87	942.59	861.61	3999.35	T/O	T/O	OOM	–
	ARI↑	8.8 $\pm$ 3.3	26.4 $\pm$ 3.6	33.7 $\pm$ 9.2	3.1 $\pm$ 1.9	-7.6 $\pm$ 2.9	22.1 $\pm$ 9.7	41.7 $\pm$ 1.7	T/O	T/O	OOM	–
FCE [10]	time↓	1.27	1.34	3.60	10.88	7.93	9.92	242.65	55.48	90.79	393.27	81.71
	ARI↑	4.7 $\pm$ 2.1	32.1 $\pm$ 4.5	1.2 $\pm$ 1.2	1.2 $\pm$ 0.9	0.1 $\pm$ 0.1	11.3 $\pm$ 5.8	36.1 $\pm$ 1.6	1.3 $\pm$ 0.2	0.0 $\pm$ 0.0	1.3 $\pm$ 1.1	8.9 $\pm$ 1.7
TBAWEC [37]	time↓	30.47	66.13	249.51	2461.23	1378.39	3077.75	T/O	T/O	T/O	T/O	–
	ARI↑	11.5 $\pm$ 2.5	<u>35.3</u> $\pm$ 3.3	<u>54.9</u> $\pm$ 29.6	<u>5.6</u> $\pm$ 3.4	13.6 $\pm$ 5.4	18.5 $\pm$ 1.4	T/O	T/O	T/O	T/O	–
OLEC-SB (Ours)	time↓	0.07	0.33	0.86	1.16	1.09	1.00	17.32	6.28	12.86	244.77	28.57
	ARI↑	22.8 $\pm$ 4.8	40.7 $\pm$ 3.4	60.3 $\pm$ 13.3	12.0 $\pm$ 3.4	80.9 $\pm$ 2.2	38.7 $\pm$ 2.0	<u>44.0</u> $\pm$ 1.5	32.4 $\pm$ 7.7	79.2 $\pm$ 4.1	27.6 $\pm$ 2.4	43.9 $\pm$ 4.5

T/O indicates excessive runtime (mean execution time over 10 runs $>$ 5000 s), and OOM indicates insufficient memory. Light-gray rows denote the balance-aware baselines FCE and TBAWEC, while the dark-gray row denotes the proposed OLEC-SB.

- **Phishing Websites**³: The dataset consists of a collection of legitimate and phishing website instances. Each website is represented by the set of features which denote, whether the website is legitimate or not.
- **PSDAS**⁴: The dataset contains 4,424 records of students from a higher education institution, covering diverse undergraduate degrees such as technologies, nursing, and

management.

- **TUANDROMD**⁵: The dataset contains 4,464 instances of Android applications, categorized into malware and goodware to support security research.
- **Gender Gap**⁶: The dataset consists of 475 samples representing individual editors from the Spanish Wikipedia community.

³<http://archive.ics.uci.edu/dataset/327/phishing+websites>

⁴<https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success>

⁵<https://archive.ics.uci.edu/dataset/813/tunadromd>

⁶<https://archive.ics.uci.edu/dataset/852/gender+gap+in+spanish+wp>

TABLE S3

PERFORMANCE COMPARISON ON DIFFERENT DATASETS IN TERMS OF RUNNING TIME (SEC.) AND F-SCORE (MEAN $\pm$ STD, %). BEST RESULTS ARE HIGHLIGHTED IN **BOLD**, AND SECOND-BEST RESULTS ARE MARKED WITH AN UNDERLINE.

Method	Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	Avg.
PTGP [12]	time↓	0.04	0.20	0.65	3.06	0.36	1.95	3.72	178.95	1601.57	51.33	184.18
	F-score↑	43.1 $\pm$ 4.1	31.5 $\pm$ 1.1	57.2 $\pm$ 2.1	39.9 $\pm$ 1.2	76.3 $\pm$ 7.0	53.4 $\pm$ 2.8	25.3 $\pm$ 1.4	38.8 $\pm$ 2.5	87.4 $\pm$ 0.6	48.4 $\pm$ 3.1	50.1 $\pm$ 2.6
SEC [31]	time↓	0.00	0.01	0.01	0.03	0.01	0.03	0.79	0.16	0.13	0.47	0.16
	F-score↑	49.8 $\pm$ 3.4	31.7 $\pm$ 2.6	60.6 $\pm$ 2.3	<u>50.4</u> $\pm$ 3.4	74.0 $\pm$ 8.6	57.1 $\pm$ 3.8	39.1 $\pm$ 1.2	47.4 $\pm$ 10.6	91.3 $\pm$ 5.9	<u>53.6</u> $\pm$ 2.9	<u>55.5</u> $\pm$ 4.5
LWEA [6]	time↓	0.25	0.30	1.14	3.31	3.61	3.28	5.51	15.16	53.73	152.54	23.88
	F-score↑	49.8 $\pm$ 1.8	33.6 $\pm$ 2.3	61.5 $\pm$ 0.5	41.8 $\pm$ 2.4	<u>80.0</u> $\pm$ 2.4	61.9 $\pm$ 0.3	33.7 $\pm$ 1.2	<u>47.9</u> $\pm$ 7.4	<u>92.9</u> $\pm$ 3.5	48.9 $\pm$ 2.3	55.2 $\pm$ 2.4
LWGP [6]	time↓	0.09	0.16	0.19	0.33	0.28	0.36	1.57	0.88	1.38	3.17	0.84
	F-score↑	50.2 $\pm$ 4.4	35.8 $\pm$ 1.7	61.7 $\pm$ 0.0	40.6 $\pm$ 1.5	77.9 $\pm$ 3.3	61.7 $\pm$ 0.4	40.2 $\pm$ 0.7	45.4 $\pm$ 5.6	90.9 $\pm$ 0.2	49.9 $\pm$ 0.1	55.4 $\pm$ 1.8
ECPCS-HC [17]	time↓	0.22	0.70	1.19	3.62	3.72	14.21	17.69	45.00	67.77	262.42	41.66
	F-score↑	51.2 $\pm$ 0.6	27.0 $\pm$ 0.8	60.5 $\pm$ 2.6	40.9 $\pm$ 1.8	77.0 $\pm$ 4.1	61.8 $\pm$ 0.2	29.5 $\pm$ 1.7	41.1 $\pm$ 1.1	91.5 $\pm$ 2.8	45.9 $\pm$ 2.8	52.6 $\pm$ 1.9
ECPCS-MC [17]	time↓	0.09	0.17	0.32	0.73	0.73	1.21	3.87	5.08	4.59	12.55	2.93
	F-score↑	52.3 $\pm$ 1.8	27.6 $\pm$ 0.9	57.7 $\pm$ 3.4	41.0 $\pm$ 1.2	79.1 $\pm$ 0.5	61.7 $\pm$ 0.0	36.1 $\pm$ 1.1	38.3 $\pm$ 2.7	89.7 $\pm$ 0.6	45.4 $\pm$ 3.0	52.9 $\pm$ 1.5
U-SENC [13]	time↓	0.02	0.04	0.06	0.13	0.10	0.13	0.85	0.47	0.85	1.71	0.43
	F-score↑	46.2 $\pm$ 5.9	33.0 $\pm$ 0.8	58.9 $\pm$ 3.0	40.2 $\pm$ 0.9	73.8 $\pm$ 7.3	60.3 $\pm$ 2.2	41.2 $\pm$ 1.0	39.1 $\pm$ 2.8	89.9 $\pm$ 0.4	48.1 $\pm$ 3.1	53.1 $\pm$ 2.7
ECCMS [7]	time↓	9.35	11.99	67.75	340.66	424.44	401.62	1422.21	4245.39	T/O	T/O	–
	F-score↑	51.0 $\pm$ 1.9	33.1 $\pm$ 2.4	61.2 $\pm$ 1.7	41.8 $\pm$ 4.0	79.9 $\pm$ 3.0	<u>62.3</u> $\pm$ 2.2	27.2 $\pm$ 3.2	47.4 $\pm$ 4.1	T/O	T/O	–
FSEC [22]	time↓	0.27	0.50	0.72	13.94	13.93	1.49	3.05	5.43	22.54	21.20	8.31
	F-score↑	44.3 $\pm$ 4.9	39.3 $\pm$ 2.5	55.6 $\pm$ 3.4	44.6 $\pm$ 1.1	92.2 $\pm$ 0.9	57.3 $\pm$ 5.4	37.6 $\pm$ 0.7	41.5 $\pm$ 7.3	84.4 $\pm$ 2.2	48.2 $\pm$ 1.7	54.5 $\pm$ 3.0
YACHT [21]	time↓	1.56	1.14	3.72	20.18	12.83	10.18	23.18	94.48	81.06	178.68	36.70
	F-score↑	43.8 $\pm$ 3.3	38.9 $\pm$ 2.7	65.8 $\pm$ 7.6	40.8 $\pm$ 2.5	66.8 $\pm$ 5.0	57.7 $\pm$ 5.5	44.9 $\pm$ 1.5	43.6 $\pm$ 6.9	65.4 $\pm$ 2.9	48.1 $\pm$ 4.2	51.5 $\pm$ 4.2
SDGCA [5]	time↓	27.87	25.13	121.11	601.67	1283.63	1250.41	4525.16	T/O	T/O	OOM	–
	F-score↑	51.2 $\pm$ 3.8	38.4 $\pm$ 1.6	61.7 $\pm$ 0.0	41.3 $\pm$ 2.5	79.0 $\pm$ 0.1	60.7 $\pm$ 1.4	27.0 $\pm$ 4.1	T/O	T/O	OOM	–
GPEC [8]	time↓	62.97	100.08	231.57	777.87	942.59	861.61	3999.35	T/O	T/O	OOM	–
	F-score↑	41.3 $\pm$ 0.9	36.2 $\pm$ 2.8	67.8 $\pm$ 5.0	39.1 $\pm$ 0.8	62.3 $\pm$ 1.7	54.7 $\pm$ 5.2	42.4 $\pm$ 1.7	T/O	T/O	OOM	–
FCE [10]	time↓	1.27	1.34	3.60	10.88	7.93	9.92	242.65	55.48	90.79	393.27	81.71
	F-score↑	36.8 $\pm$ 1.4	39.6 $\pm$ 4.0	50.8 $\pm$ 0.6	36.5 $\pm$ 0.5	57.6 $\pm$ 0.0	45.8 $\pm$ 3.5	36.8 $\pm$ 1.5	26.1 $\pm$ 0.2	62.5 $\pm$ 0.0	36.6 $\pm$ 0.7	42.9 $\pm$ 1.3
TBAWEC [37]	time↓	30.47	66.13	249.51	2461.23	1378.39	3077.75	T/O	T/O	T/O	T/O	–
	F-score↑	41.3 $\pm$ 1.7	<u>44.6</u> $\pm$ 2.9	<u>77.6</u> $\pm$ 14.7	39.3 $\pm$ 2.2	63.0 $\pm$ 2.3	50.1 $\pm$ 0.8	T/O	T/O	T/O	T/O	–
OLEC-SB (Ours)	time↓	0.07	0.33	0.86	1.16	1.09	1.00	17.32	6.28	12.86	244.77	28.57
	F-score↑	<u>51.8</u> $\pm$ 3.2	47.3 $\pm$ 3.0	80.2 $\pm$ 6.6	51.0 $\pm$ 2.2	93.4 $\pm$ 0.8	63.4 $\pm$ 1.4	<u>44.6</u> $\pm$ 1.5	70.4 $\pm$ 2.9	96.7 $\pm$ 0.9	60.5 $\pm$ 1.0	65.9 $\pm$ 2.4

T/O indicates excessive runtime (mean execution time over 10 runs > 5000 s), and OOM indicates insufficient memory. Light-gray rows denote the balance-aware baselines FCE and TBAWEC, while the dark-gray row denotes the proposed OLEC-SB.

- **COIL100**⁷: The dataset contains 7,200 color images of 100 different objects, with 72 images per object taken from various angles.
- **Crowdsourced Mapping**⁸: The dataset includes 29 features, such as NDVI (Normalized Difference Vegetation Index) values and other spectral data extracted from multiple timestamps across different seasons.
- **HTRU2**⁹: The dataset contains 17,898 samples, each representing a candidate for a pulsar signal identified during the High Time Resolution Universe Survey.
- **Steel Energy**¹⁰: The dataset consists of 35,040 instances recorded at 15-minute intervals over a one-year period from a steel plant in South Korea.

B. Compared Methods

We compared our method with the following methods.

- **PTGP** [12]: this method builds a microcluster-cluster bipartite graph, assigns weights using dense similarities

from random walk probability trajectories on a sparse graph, and then performs graph partitioning.

- **SEC** [31]: this method performs spectral clustering on the co-association matrix and accelerates it via the theoretical equivalence to weighted k -means.
- **LWEA and LWGP** [6]: these methods estimate cluster reliability via the ensemble-driven cluster index (ECI). LWEA builds a locally weighted co-association matrix and applies hierarchical clustering, while LWGP integrates ECI into sample-cluster link weights to construct a locally weighted bipartite graph for partitioning.
- **ECPCS-HC and ECPCS-MC** [17]: these methods enhance co-association matrix by exploiting multiscale cluster similarities derived from random walks. ECPCS-HC transfers cluster-level similarities to the sample level to build an enhanced co-association matrix and applies hierarchical clustering, while ECPCS-MC partitions the cluster similarity graph into meta-clusters via normalized-cut and assigns samples by majority voting.
- **U-SENC** [13]: this method integrates multiple ultra-scalable spectral clustering to generate the ensemble and derives the consensus by partitioning a sample-cluster

⁷<https://www.cs.columbia.edu/CAVE/software/softlib/coil-100.php>

⁸<https://archive.ics.uci.edu/dataset/400/crowdsourced+mapping>

⁹<https://archive.ics.uci.edu/dataset/372/htru2>

¹⁰<https://archive.ics.uci.edu/dataset/851/steel+industry+energy+consumption>

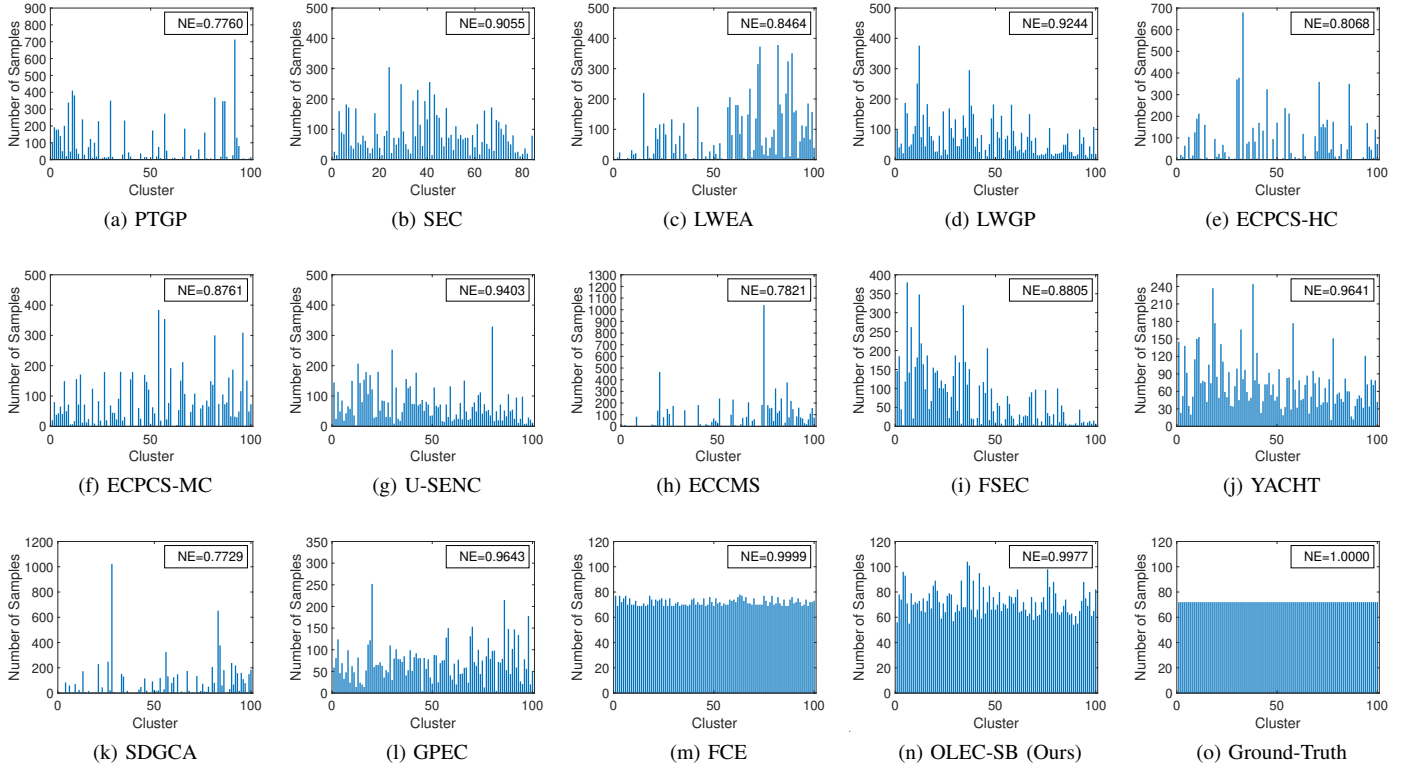


Fig. S1. Distribution of samples in each cluster on the D7 dataset in a single experimental run.

TABLE S4
THE AVERAGE RANK AR_i OF EACH METHOD ON DATASETS D1-D6.

	PTGP	SEC	LWEA	LWGP	ECPCS-HC	ECPCS-MC	U-SENC	ECCMS	FSEC	YACHT	SDGCA	GPEC	FCE	TBAWEC	OLEC-SB
ACC	12.33	8.83	6.67	7.33	8.33	7.50	9.67	6.83	5.83	7.67	6.33	10.00	11.83	8.83	2.00
ARI	11.00	10.17	7.00	9.00	10.50	8.00	7.50	8.50	7.17	5.67	8.67	9.83	10.50	5.50	1.00
F-score	12.00	8.50	5.33	6.83	8.00	7.17	10.00	6.67	7.17	8.33	5.83	10.33	13.00	9.67	1.17

bipartite graph.

- **ECCMS** [7]: this method extracts high-confidence information from base clusterings to form a sparse matrix and employs a self-enhancement mechanism to propagate reliable information while denoising the co-association matrix.
- **FSEC** [22]: this method constructs an anchor graph via fast k -nearest neighbor approximation, generates base clusterings using repeated k -means on spectral embeddings from a single singular value decomposition, and applies transfer cut on the resulting bipartite graph.
- **YACHT** [21]: this method converts base clusterings into a hypergraph using random walks, reconstructs it via matrix factorization with hypergraph regularization to learn anchors and anchor similarities, and derives the consensus from the learned anchor similarity matrix.
- **SDGCA** [5]: this method incorporates cluster-size information via normalized ensemble entropy to construct a weighted co-association matrix, explores high-order proximity using random walks, and refines similarity and dissimilarity matrices through adversarial learning for

hierarchical clustering.

- **GPEC** [8]: this method derives a bias-diversity decomposition from generalization theory, approximates the expected co-association matrix using high-confidence second-order similarities, and learns adaptive weights via min-max optimization followed by spectral embedding and k -means.
- **FCE** [10]: this method introduces a regularization term measuring fairness and cluster capacity equality and integrates it into an ensemble clustering framework optimized iteratively.
- **TBAWEC** [37]: this method formulates ensemble clustering as an optimization problem with normalized similarity matrices and exclusive lasso regularization, and jointly learns adaptive weights and discrete cluster indicators via alternating optimization.

C. Clustering Performance Comparison

In the supplemental material, we further present the results in terms of ARI and F-score. From Table S2, OLEC-SB achieves the best ARI on the majority of datasets, except for D7, where it attains the second-best result. From Table S3

OLEC-SB also achieves the best F-score on most datasets, with second-best performance on D1 and D7.

D. Balance Performance

Figure S1 presents the cluster-size distribution on the D7 dataset for different ensemble clustering methods.

E. Statistical Analysis

Friedman testing [1] is conducted to examine the statistical significance of clustering performance *w.r.t.* fifteen methods in terms of ACC, ARI and F-score. Under the null hypothesis, all methods are presumed to be equivalent; thus, rejecting the null hypothesis indicates statistically significant differences among the compared methods. The average rank AR_i is calculated by

$$AR_i = \frac{1}{N_F} \sum_j r_{ij}, \quad (S25)$$

where r_{ij} denotes the rank of the i -th method on the j -th dataset. The average rank of the compared methods on ACC, ARI and F-score are summarized in Table S4. The τ_F of Friedman testing is calculated using Eqs. (S26) and (S27):

$$\tau_F = \frac{(N_F - 1)\tau_{\chi^2}}{N_F(K_F - 1) - \tau_{\chi^2}}, \quad (S26)$$

$$\tau_{\chi^2} = \frac{12N_F}{K_F(K_F + 1)} \left(\sum_{i=1}^{K_F} AR_i^2 - \frac{K_F(K_F + 1)^2}{4} \right), \quad (S27)$$

where K_F denotes the number of methods and N_F denotes the number of datasets. In our setting, $K_F = 15$ methods are evaluated on $N_F = 6$ datasets (D1-D6). The Friedman statistic τ_F follows an F distribution with degrees of freedom $(K_F - 1)$ and $(K_F - 1)(N_F - 1)$. At the 95% confidence level, the critical value is $F_{0.05}((K_F - 1), (K_F - 1)(N_F - 1)) = F_{0.05}((15 - 1), (15 - 1)(6 - 1)) = 1.8357$. Since this critical value is much smaller than the computed τ_F values (2.3362 for ACC, 2.5224 for ARI, 3.5685 for F-score), we conclude that the differences among the fifteen methods are statistically significant at the 5% level.

REFERENCES

- [1] Z. Deng, L. Liang, H. Yang, W. Zhang, Q. Lou, K.-S. Choi, T. Zhang, J. Zhou, and S. Wang, "Enhanced multiview fuzzy clustering using double visible-hidden view cooperation and network lasso constraint," *IEEE Transactions on Fuzzy Systems*, vol. 30, no. 11, pp. 4965–4979, 2022.